Table des matières

Introduction **4**

1 Les réseaux IOT : Concepts Fondamentaux **5**

1.1 Introduction 6
1.2 L'Internet des objets (IoT) 6
1.3 Architecture des réseaux IoT 8
1.3.1 Qian Xiaocong, Zhang Jidong Architecture 8
1.3.2 ITU (Union Internationale des Télécommunications) Architecture 9
1.4 Technologie et outils de l'IOT : 11
1.4.1 Technologies d'identification et de communication : 11
1.5 Caractéristiques des nœuds de capteurs IoT 14
1.6 Défis de sécurité des réseaux IOT 15
1.6.1 Types d'attaques connues dans l'IoT 16
1.6.2 Mécanismes de sécurité 17
1.7 Conclusion 19

2 Étude des systèmes de détection des intrusions : IDS **20**

2.1 Introduction 21
2.2 Définition de l'IDS 21
2.3 Caractéristiques d'un système de détection d'intrusion 22
2.4 Différents types d'IDS 22
2.5 Architecture des IDS 25
2.6 Vocabulaire (notions) utilisé dans la détection d'intrusions 26
2.7 Emplacement d'un système de détection d'intrusions 27
2.8 Classification des IDS 28
2.8.1 Sources et nature des données à analyser 28
2.8.2 Méthodes de détection 28
2.8.2.1 Approche par signature 29
2.8.2.2 Approche comportementale (détection d'anomalies) 30
2.8.2.3 Approche spécifications 30
2.8.3 Lieu de l'analyse des données 30

2.8.4 Fréquence de l'analyse 31
2.8.5 Comportement en cas de détection d'une attaque 31
2.9 Mesures performance des IDS : 32
2.10 Limites et vulnérabilités des IDS 34
2.11 Les systèmes de prévention d'intrusion (IPS) 34
2.12 Similarités et différences entre IDS et IPS 34
2.12.1 Similarités 34
2.12.2 Différences 36
2.13 Exemples des IDS existants : 36
2.13.1 Survey on Federated Learning for Intrusion Detection System : Concept, Architectures, Aggregation Strategies, Challenges, and Future Directions [1] 36
2.13.2 Toward Enhancing Privacy Preservation of a Federated Learning CNN Intrusion Detection System in IoT : Method and Empirical Study [2] . . 37
2.13.3 Federated Learning for IoT Intrusion Detection [3] 37
2.13.4 An optimal federated learningbased intrusion detection for IoT environment [4] 38
2.14 Conclusion 39

3 Solution proposée 40
3.1 Introduction 41
3.2 Introduction à l'intelligence artificielle : 41
3.2.1 Apprentissage automatique 41
3.2.1.1 Types d'apprentissage (Types of Learning) 41
3.2.2 Apprentissage en profondeur(Deep learning) 43
3.2.2.1 Réseau de neurones profond DNN 43
3.2.2.2 Convolutional neural networks (CNNs) 44
3.2.2.3 Les réseaux de neurones récurrents (RNN) 44
3.2.3 Détection d'anomalies : 46
3.2.3.1 État de l'art : 46
3.3 Modèle proposé : AutoEncodeur + MLP + FedDistillation : 46
3.3.1 Autoencodeur 46
3.3.1.1 Architecture generale : 46
3.3.1.2 Les Hyperparamètres 47
3.3.1.3 Variantes des Autoencodeurs 48
3.3.2 MLP : 50
3.3.2.1 Definition au MLP : 50
3.3.2.2 Architecture du MLP : 50
3.3.2.3 Fonctionnement du MLP : 51

3.3.2.4 Paramètres et optimisation : 52
3.3.2.5 Avantages et limites du MLP : 53
3.3.3 Federated Learning : 54
3.3.3.1 Introduction au Federated Learning : 54
3.3.3.2 Fonctionnement du Federated Learning : 55
3.3.3.3 Algorithmes et architectures du Federated Learning : 55
3.3.3.4 Paramètres et optimisation : 55
3.3.3.5 Avantages du FL : 55
3.3.3.6 Limites et défis : 55

4 Implémentation et résultats 56
4.1 Introduction 56
4.2 Implémentation 56
4.3 Environnement de développement 56
4.3.1 Google Colab : 56
4.3.2 Visual Studio Code : 56
4.4 Langage de programmation et bibliothèques 56
4.4.1 Python : 56
4.4.2 Bibliothèques utilisées : 57
4.5 Base d'apprentissage et de test : CICIoT2023 – TON$_{I}oT\breve{}BoT_{I}oT\breve{}NSL_{K}DD$. 58
4.5.1 NSL-KDD : 58
4.5.2 CICIoT-2023 : 59
4.5.3 BoT-IoT : 59
4.6 Implémentation de l'architecture proposée 59
4.6.1 Phase de prétraitement 59
4.6.2 Phase de création du modèle et d'apprentissage 59
4.6.3 Phase de prédiction et d'évaluation 59
4.7 Résultats, discussions et comparaison 59
4.8 Conclusion 59

Introduction

Contexte général

Problématique

Objectifs

Méthodologie

Plan du mémoire

Ce mémoire est structuré comme suit :

— Le premier chapitre présente les généralités sur les IDS.

— Le deuxième chapitre traite de l'état de l'art et des études récentes.

— Enfin, une conclusion générale résume les résultats.

Chapitre 1

Les réseaux IOT : Concepts Fondamentaux

1.1 Introduction

Depuis quelques années, les réseaux IoT ne cessent d'augmenter et de se développer. De nouveaux objets connectés apparaissent et les technologies associées évoluent également. Ce nouveau paradigme représente une évolution de l'Internet, permettant de connecter les objets de notre quotidien à Internet ainsi qu'entre eux.
L'IoT possède plusieurs définitions qui seront présentées plus tard dans ce chapitre, mais d'une manière générale, il est défini comme une deuxième version d'Internet, conçue non pas uniquement pour les humains, mais aussi pour les objets connectés.
Ce chapitre propose une vue d'ensemble des concepts fondamentaux liés à l'IoT. Il commence par présenter différentes définitions, puis aborde les principales caractéristiques ainsi que l'architecture des réseaux IoT, avant de se conclure par une analyse des défis de sécurité auxquels ces systèmes sont confrontés.

1.2 L'Internet des objets (IoT)

L'Internet des objets (IoT) consiste à connecter des objets physiques, tels que des capteurs et des appareils, à Internet. Ces appareils sont dotés de capacités de communication leur permettant d'échanger des données entre eux et avec leur environnement via le réseau, sans intervention humaine. Les réseaux IoT sont dynamiques, les appareils les rejoignent et les quittent fréquemment. Les applications IoT sont de plus en plus utilisées dans l'industrie, la santé, l'agriculture, les transports et la gestion de l'énergie.[1].

L'Internet des objets (IoT) est défini par diverses organisations internationales, notamment l'ISO/IEC, l'IETF, l'IRTF et l'IEEE, comme un réseau d'objets physiques capables de collecter et de transmettre des données. Ces objets interconnectés, accompagnés d'humains et de ressources informa tionnelles, forment une infrastructure permettant le traitement des informations et l'adaptation des actions en conséquence.

Les exigences de l'ISO/IEC pour l'IoT incluent l'auto-configuration, l'identification unique, la mobilité, la connectivité, la fiabilité et la standardisation des interfaces. L'IETF met en avant l'utilisation de protocoles standardisés pour garantir la communication entre les objets. De son côté, le groupe de recherche T2TRG de l'IRTF définit l'IoT comme un réseau où des nœuds à faibles ressources interagissent avec Internet pour favoriser l'innovation.

L'IEEE, à travers le projet P2413, ainsi que des acteurs industriels tels que Cisco, IBM, Huawei et Intel, contribuent à structurer l'IoT en tant que système interconnecté, adapté aux exigences spécifiques des différents secteurs. Le NIST considère l'IoT comme un système cyber physique améliorant la qualité de vie via des objets intelligents, tandis que l'OASIS insiste sur l'adaptation des technologies existantes pour une meilleure intégration des capteurs et des réseaux physiques.

En résumé, l'IoT repose sur la connectivité des objets à Internet afin de proposer des services

accessibles partout et à tout moment, en s'appuyant sur des standards et protocoles rigoureux pour garantir son efficacité et son évolution.[2]

1.3 Architecture des réseaux IoT

L'un des problèmes liés à l'IoT est qu'il s'agit d'un concept très vaste et général, pour lequel aucune architecture uniforme n'a été proposée.[3] Cependant, certaines architectures ou modèles IoT ont été présentés par différents chercheurs, auteurs et praticiens :

1.3.1 Qian Xiaocong, Zhang Jidong Architecture

Selon Qian Xiaocong et Zhang Jidong (2012), l'IoT traditionnel est composé de trois couches.

— **Couche de perception :** La couche de perception est la couche la plus basse de l'architecture IoT. Elle est responsable de la collecte des données du monde réel. Les capteurs et actionneurs sans fil sont les principaux composants de cette couche.

— **Couche transport(réseau) :** ccupant une position intermédiaire dans l'architecture de l'IoT, elle joue un rôle essentiel en assurant la transmission des données entre la couche de perception et la couche application. Parmi les technologies les plus couramment utilisées à ce niveau figurent Bluetooth Low Energy (BLE), Zigbee, WiFi, LoRaWAN (Long Range Wide Area Networks) et IPv6, notamment via les réseaux personnels sans fil à faible consommation (6LoWPAN).

— **Couche d'application :** La couche Application représente l'aspect le plus visible de l'architecture de l'Internet des Objets. Elle regroupe l'ensemble des applications qui exploitent les données collectées par la couche Perception. Son rôle essentiel consiste à agréger, analyser et interpréter ces données afin d'offrir une variété de services adaptés aux besoins des utilisateurs. Utilisation de protocoles tels que MQTT, CoAP ou HTTP pour garantir un transfert de données efficace.

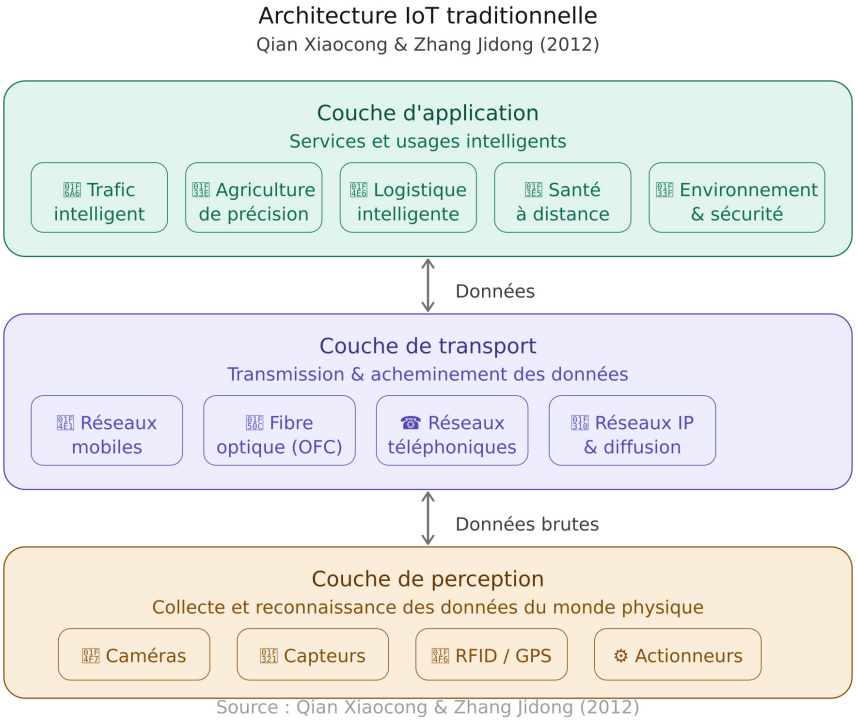

FIGURE 1.1 – Qian Xiaocong, Zhang Jidong Architecture

1.3.2 ITU (Union Internationale des Télécommunications) Architecture

Selon les recommandations de l'Union Internationale des Télécommunications (UIT), l'architecture réseau de l'Internet des Objets se compose de 5 couches :

— **Sensing Layer (Couche de perception)**
Il s'agit de la couche physique. Son rôle est d'identifier les objets intelligents dans l'environnement et de collecter des informations en détectant divers paramètres physiques grâce à des capteurs.

— **Access Layer (Couche d'accès)**
Cette couche gère la connectivité initiale des objets. Elle permet aux données de quitter la couche physique, souvent en passant par des passerelles (gateways), pour être acheminées vers les infrastructures de traitement

— **Network Layer (Couche réseau)**
Elle est responsable de la connexion avec d'autres objets intelligents, des dispositifs réseau et des serveurs. Son rôle principal est d'assurer la transmission et le traitement des données provenant des capteurs

— **Middleware Layer (Couche Middleware)**
Cette couche logicielle agit comme un pont. Son rôle est de transformer les données brutes en informations exploitables en utilisant des outils de stockage à la demande et

des capacités d'analyse (Cloud et Big Data). Elle peut intégrer de l'intelligence artificielle pour créer de la valeur à partir des données analysées.

— **Application Layer (Couche d'application)**
Elle fournit des services spécifiques à l'utilisateur final. Elle définit les domaines d'application concrets de l'IoT, tels que les maisons intelligentes, les villes intelligentes ou la santé connectée

Ces couches sont similaires au modèle de référence OSI (Open Systems Interconnection) utilisé en réseau et en communication de données.

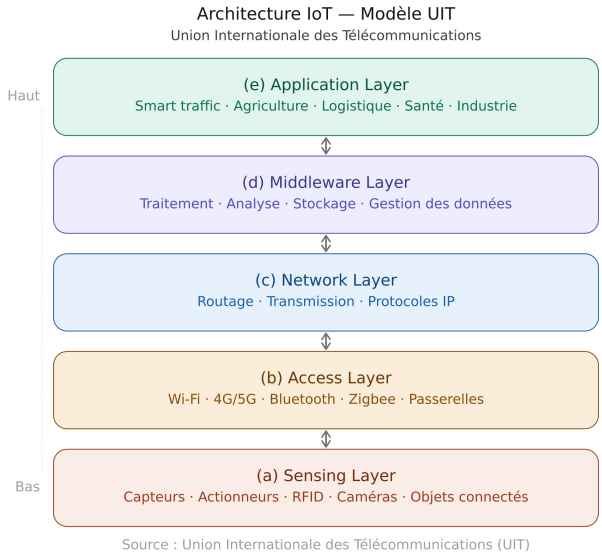

FIGURE 1.2 – ITU Architecture

1.4 Technologie et outils de l'IOT :

1.4.1 Technologies d'identification et de communication :

a- **Technologies d'identification :**

Ces technologies permettent d'identifier de manière unique les objets dans un réseau IoT.

- **RFID (Radio Frequency Identification)**
  Les technologies d'identification par radiofréquences (RFID), permettent de transmettre l'identité de tout objet équipé d'une étiquette RFID à distance, à L'aide d'ondes radio. Cette étiquette peut être lue par un lecteur externe qui collecte et transmet les informations contenues dans l'étiquette.
- **EPC (Electronic Product Code)**
  C'est un code électronique de 64 ou 98 bits enregistré électroniquement sur une étiquette RFID. l'EPC est une technologie conçue pour identifier chaque objet de manière unique
- **NFC (Near-field communication)**
  La communication en champ proche (NFC) est une technologie sans fil à courte portée qui utilise l'induction de champ magnétique pour permettre la communication entre les appareils lorsqu'ils sont en contact ou à quelques centimètres l'un de l'autre. Cela englobe des fonctionnalités telles que l'authentification des cartes de crédit, l'activation d'accès physique, le transfert de petits fichiers et l'établissement de connexions sans fil plus rapides. En général, NFC repose sur les écosystèmes et les normes existants autour des étiquettes d'identification par radiofréquence (RFID) et étend leurs capacités.

b- **Technologies de communication :**

Elles permettent le transfert des données entre les objets IoT.

- **WIFI**
  Le Wi-Fi est une technologie de réseau sans fil qui permet aux périphériques tels que des ordinateurs, des périphériques mobiles, et d'autres équipements (imprimantes et caméras vidéo) d'accéder à Internet. Il permet à ces appareils, et à de nombreux autres, d'échanger des renseignements entre eux, ce qui crée un réseau qui peut prendre en charge de nombreux appareils simultanément. Il est notamment utilisé dans les maisons intelligentes, les villes intelligentes et dans le domaine de l'industrie.
- **Bluetooth**
  désigne une norme de communication sans fil par ondes radio capable de transmettre des données et de la voix entre deux appareils électroniques compatibles. Il fonctionne sur les bandes de fréquence 2,4 GHz avec une portée maximale qui varie de 10 à 100 mètres.

— **BLE :**

Il s'agit d'une technologie de réseau personnel sans fil utilisée pour les commu nications à courte portée. Cette technologie à faible consommation d'énergie trouve de nombreuses applications dans de nombreux contextes IoT, notamment, mais sans s'y limiter, les environnements résidentiels intelligents, les infrastructures urbaines intelli gentes et les systèmes de santé. La principale différence entre BLE et Bluetooth est que le BLE est conçu pour une faible consommation d'énergie.

— **Zigbee** Zigbee est un protocole de communication sans fil basé sur la norme IEEE 802.15.4, conçu pour les applications à faible consommation d'énergie et à faible débit de données. Largement utilisé dans l'IoT, il repose sur une topologie en réseau maillé permettant à chaque appareil de relayer les données, ce qui assure la robustesse et la continuité du réseau en cas de panne d'un nœud. Son interopérabilité entre appareils de différents fabricants est garantie par la Zigbee Alliance.

c- **Protocoles de communication IoT :** Ces protocoles définissent comment les données sont échangées.

— **MQTT (Message Queuing Telemetry Transport)**
C'est un protocole de communication léger basé sur le modèle Publisher/Subscriber où les objets connectés envoient leurs données à un broker qui les redistribue aux appareils abonnés.

— **IEEE 802.15.4**
Le protocole IEEE 802.15.4 est une norme définissant les couches physique et MAC pour les réseaux sans fil à faible débit (LR-WPAN), conçue pour des dispositifs à faible consommation d'énergie et courte portée. Il supporte des débits allant jusqu'à 850 kb/s et intègre des mécanismes MAC tels que le CSMA/CA, la communication sécurisée et la gestion de la qualité du lien. Ce protocole constitue la base technologique de nombreuses piles IoT, notamment ZigBee, WirelessHART, 6LoWPAN et RPL, ce qui en fait un standard fondamental pour l'interconnexion des objets intelligents.

— **6LoWPAN (IPv6 over Low power Wireless Personal Area Networks)**
Le protocole 6LoWPAN est une couche d'adaptation permettant aux paquets IPv6 de circuler sur des réseaux sans fil à faible consommation basés sur IEEE 802.15.4. Son rôle principal est de connecter des objets contraints directement à Internet tout en supportant leur mobilité. 6LoWPAN constitue un pont technologique essentiel entre le protocole Internet standard et les contraintes physiques des objets connectés à faible puissance.

— **CoAP (Constrained Application Protocol)**
Le protocole CoAP est une norme de couche application conçue spécifiquement pour l'Iot Standardisé par l'IETF, il a été développé pour répondre aux besoins des réseaux contraints et à faible consommation, tels que les LLNs (Low Power and Lossy Networks) et 6LoWPAN. CoAP est conçu comme une version simplifiée et légère de HTTP. Il utilise le même modèle d'interaction requête/réponse entre applications, ce qui facilite grandement l'intégration des réseaux d'objets connectés avec le web traditionnel, mais à l'inverse de HTTP qui repose sur TCP, CoAP fonctionne sur UDP.

1.5 Caractéristiques des nœuds de capteurs IoT

Les nœuds de capteurs qui constituent la base des réseaux IoT sont des composants électroniques miniaturisés conçus pour être intégrés dans des objets physiques dans le but de collecter des informations spécifiques et de les communiquer entre eux et/ou avec la station de base. Pour assurer ces tâches, chaque nœud de capteurs est généralement composé des éléments suivants :

— **Unité d'acquisition :**
Elle mesure des grandeurs physiques comme l'humidité, la température ou la pression.

— **Unité de traitement :**
Elle collecte, vérifie et stocke les données, puis les prépare pour la transmission. Cette unité dispose de capacités limitées en termes de calcul, de mémoire et de traitement, et ces ressources ne sont généralement pas extensibles.

— **Unité de communication :**
Constituée d'un émetteur et d'un récepteur, elle permet l'échange de données avec la station de base et les autres nœuds via des liaisons radio, filaires ou non filaires. Sa portée étant restreinte, une antenne externe peut être nécessaire pour améliorer la transmission.

— **Source d'énergie :**
Une batterie alimente l'ensemble des composants. La durée de fonctionnement du nœud dépend directement de sa capacité énergétique.

— **Système de localisation :**
Basé sur le GPS ou un algorithme interne, il permet de situer géographiquement le capteur.

— **Unité de mobilité :**
Permet au nœud de se déplacer pour exécuter ses tâches, elle dispose de sa propre alimentation indépendante de la batterie principale.

En conséquent, les nœuds de capteurs sont caractérisés par :

— Une capacité de stockage et de traitement très restreinte.

— Une courte portée de transmission liée aux limitations de la puissance du signal et de la capacité de rayonnement des antennes utilisées.

— Une énergie limitée : En général, les batteries utilisées ont une capacité énergétique très limitée. De plus, dans la plupart des applications, il est problématique, voire impossible, de recharger ou de changer la batterie. Et lorsque cette dernière est épuisée, le nœud de capteurs ne peut fournir aucun service et il est considéré mort et il est instantanément retiré du réseau

1.6 Défis de sécurité des réseaux IOT

L'Internet des objets (IoT) connecte un grand nombre d'équipements hétérogènes à Internet. Cette interconnexion massive expose ces systèmes à de nombreuses menaces de sécurité. Et contrairement au réseau classique les caractéristiques des réseaux IOT ont engendré des contraintes spécifiques : celui des ressources limitées en calcul, mémoire et énergie, qui caractérisent ces dispositifs.

Ces contraintes représentent un enjeu central, notamment en matière de sécurité, où la garantie de la confidentialité et de l'intégrité des données dépend fortement des capacités matérielles des appareils.
Les dispositifs IoT sont conçus pour être efficaces, compacts et peu énergivores, mais cette conception les rend vulnérables à diverses menaces, notamment à cause de leur incapacité à exécuter des algorithmes cryptographiques complexes.
De plus, leur mémoire limitée complique la gestion et le stockage sécurisé des clés, accentuant les risques d'accès non autorisé, surtout dans des environnements distants ou non surveillés.
L'approvisionnement énergétique restreint, souvent basé sur des batteries à longue durée de vie, ajoute une contrainte supplémentaire, car les mécanismes de sécurité énergivores peuvent compromettre leur autonomie. Ainsi, dans le contexte de la cybersécurité IoT, prendre en compte ces limitations matérielles n'est pas un simple choix d'optimisation, mais une exigence fondamentale.

1.6.1 Types d'attaques connues dans l'IoT

— **Attaques physique et altération matérielle**
Dans un environnement non sécurisé, les dispositifs IoT peuvent être accessibles depuis l'extérieur, ce qui les rend vulnérables aux attaques physiques. Ces attaques consistent à manipuler directement le matériel afin d'altérer son fonctionnement ou d'extraire des données sensibles.

Avec la multiplication des objets connectés, les attaques d'altération matérielle deviennent de plus en plus fréquentes, en raison de la valeur des équipements et de leur grande disponibilité. Par conséquent, la sécurité physique des dispositifs IoT constitue un enjeu majeur et doit être considérée comme une priorité pour les organisations.

— **Attaques par déni de service (DoS/DDoS) :**
Un appareil IoT est submergé de requêtes frauduleuses jusqu'à épuisement de ses ressources (CPU, batterie). Par exemple,le botnet Mirai a exploité des caméras IoT pour lancer des attaques DDoS massives. Les défenses incluent la limitation du débit (rate limiting) et la détection d'anomalies.

— **Attaque Man-In-The-Middle(MitM) :**
Une attaque de type Man-In-The-Middle (MitM) consiste pour un attaquant à s'interposer entre deux entités communicantes afin d'intercepter, modifier ou manipuler les données échangées à leur insu. L'attaquant peut ainsi accéder à des informations sensibles ou altérer les communications sans être détecté.

Dans le contexte de l'IoT, ce type d'attaque est particulièrement critique en raison de l'utilisation de protocoles de communication parfois peu sécurisés et de l'absence de mécanismes de chiffrement robustes.

— **Spoofing (Attaques par usurpation) :**

La technique de « spoofing » désigne une méthode d'intrusion qui consiste à transmettre à un serveur des paquets semblant émaner d'une adresse IP reconnue par le pare-feu. Le pirate rend la machine inaccessible afin d'intercepter les codes de communication et d'établir une connexion non autorisée.

— **Attaque par Force Brute :**
Une attaque par force brute consiste à tester de manière systématique un grand nombre de combinaisons possibles (mots de passe, clés de chiffrement) jusqu'à trouver la bonne. Cette méthode repose sur l'absence ou la faiblesse des mécanismes de protection, tels que des mots de passe simples ou l'absence de limitation du nombre de tentatives.

Dans le contexte de l'IoT, ce type d'attaque est particulièrement efficace en raison de l'utilisation fréquente de configurations par défaut et de ressources limitées en matière de sécurité.

— **Sniffing (Attaque par écoute clandestine)**
Il s'agit d'écouter une ligne de transmission par laquelle circulent des données afin de les capturer en temps réel. Cette méthode peut être employée en interne pour le débogage ou de manière malveillante par un individu malintentionné cherchant, par exemple, à obtenir un mot de passe. Elle vise principalement à intercepter des données non chiffrées. Dans certains réseaux (non commutés, connectés via un concentrateur ou des câbles coaxiaux), tous les messages sont envoyés à l'ensemble des utilisateurs. En modifiant l'état de l'interface réseau (c'est-à-dire en activant le mode espion), l'attaquant peut capter toutes les communications pour les examiner.

— **Attaque par inondation de requêtes de publication MQTT :**
Une attaque ciblée sur le protocole MQTT, provoquant une inondation de requêtes de publication qui submerge le système.

1.6.2 Mécanismes de sécurité

Les mécanismes de sécurité dans les réseaux IoT ont pour objectif de protéger les dispositifs, les communications et les données contre les différentes menaces. En raison des contraintes spécifiques de l'IoT (ressources limitées, hétérogénéité), ces mécanismes doivent être adaptés et efficaces.

Voici quelques-unes des stratégies les plus efficaces :

a - **Authentification et Contrôle d'Accès** L'authentification robuste est fondamentale pour s'assurer que seuls les utilisateurs et dispositifs autorisés puissent accéder au réseau IoT. L'utilisation de mots de passe complexes et uniques, l'authentification MFA pour les dispositifs qui le supportent, et la mise en œuvre de certificats numériques pour valider l'identité des dispositifs peuvent améliorer significativement la sécurité.

b - **Chiffrement des Données** Le chiffrement des données en transit et au repos est essentiel pour protéger les informations sensibles contre l'interception ou la manipulation. Des

protocoles de communication sécurisés doivent être utilisés pour sécuriser les connexions entre les dispositifs IoT et les serveurs centraux. En plus du chiffrement en transit, le chiffrement de bout en bout est de plus en plus pertinent, garantissant que les données restent protégées tout au long de leur cycle de vie.

c - **Mises à jour du Firmware** Assurer que les dispositifs IoT reçoivent des mises à jour régulières du firmware est crucial pour atténuer les vulnérabilités connues. Les fabricants doivent fournir des mécanismes pour la mise à jour sécurisée du firmware, et les administrateurs informatiques doivent mettre en œuvre des politiques pour garantir que ces mises à jour soient appliquées en temps opportun. Les fabricants doivent concevoir des dispositifs IoT avec la capacité de recevoir des mises à jour de sécurité. Les utilisateurs, quant à eux, doivent s'assurer que leurs dispositifs sont toujours à jour.

d - **Segmentation du Réseau** La segmentation du réseau est une stratégie efficace pour limiter la portée d'une éventuelle attaque. En divisant le réseau en segments plus petits et isolés, il est possible de contenir la propagation des menaces et de protéger les actifs les plus critiques.

e - **Surveillance et Détection des Menaces** La surveillance continue des dispositifs IoT et du réseau est essentielle pour détecter des activités suspectes et répondre rapidement à d'éventuels incidents de sécurité. Mettre en œuvre des solutions qui détectent des comportements anormaux sur les dispositifs IoT permet d'identifier les attaques à leurs premiers stades. Les outils d'analyse du trafic réseau et les systèmes de détection d'intrusion (IDS) sont particulièrement utiles.

1.7 Conclusion

Ce chapitre a permis de présenter une vue d'ensemble de l'Internet des objets (IoT), en abordant ses différentes définitions, ses caractéristiques principales ainsi que ses architectures. Il a également mis en évidence les technologies et protocoles qui constituent la base de fonctionnement des systèmes IoT, tout en soulignant leur diversité et leur hétérogénéité. Une attention particulière a ensuite été portée aux défis de sécurité propres aux environnements IoT, en passant en revue les types d'attaques les plus connues ainsi que les mécanismes de sécurité existants pour y faire face. Parmi ces solutions, les systèmes de détection d'intrusion (IDS) occupent une place essentielle, dans la mesure où ils assurent une surveillance continue des dispositifs et du trafic réseau. Le chapitre suivant sera ainsi consacré à l'étude approfondie de ces systèmes.

Chapitre 2

Étude des systèmes de détection des intrusions : IDS

2.1 Introduction

Dans un contexte marqué par l'évolution de la technologie, les systèmes d'information sont aujourd'hui accessibles sur Internet, facilitant l'accès aux utilisateurs et aux internautes, et bien sûr aussi aux attaquants. Malheureusement, les anciens systèmes de prévention contre les attaques, tels que les antivirus et les pare-feu, ne sont plus efficaces face à ces nouvelles attaques.

Pour faire face à cette problématique, de nouveaux composants de sécurité sont apparus, appelés systèmes de détection et de prévention d'intrusions.

En effet, dans ce chapitre, on verra ce que c'est un IDS ainsi que son architecture. Pour détailler, on verra les techniques de détection et leur efficacité.

2.2 Définition de l'IDS

Des systèmes de détection d'intrusions ou, d'une manière générique détection d'incidents (IDS), sont nécessaires afin de renforcer la robustesse des environnements informatiques, en identifiant, le plus tôt possible, les événements non sollicités ou les incidents susceptibles de compromettre leur sécurité. [5]

Elle correspond également à la capacité d'un système de sécurité informatique à identifier automatiquement, à partir des événements liés à la sécurité, qu'une violation est en cours ou s'est produite dans le passé [6].

Un système de détection d'intrusion (IDS) repose généralement sur quatre fonctions principales :

— **Analyse :** elle consiste à examiner un grand volume de données afin d'identifier les comportements anormaux ou suspects. On utilise deux approches principales : la détection par signature et la détection par anomalie.

— **Journalisation :** c'est le fait de sauvegarder tout le trafic ou les événements dans un fichier log. tels que l'arrivée de paquets ou les tentatives de connexion.

— **Gestion :** un IDS nécessite une administration continue. Il peut être comparé à une caméra de surveillance qui doit être configurée et supervisée en permanence.

— **Action :** cette fonction permet d'alerter l'administrateur lorsqu'une activité suspecte ou une attaque est détectée.

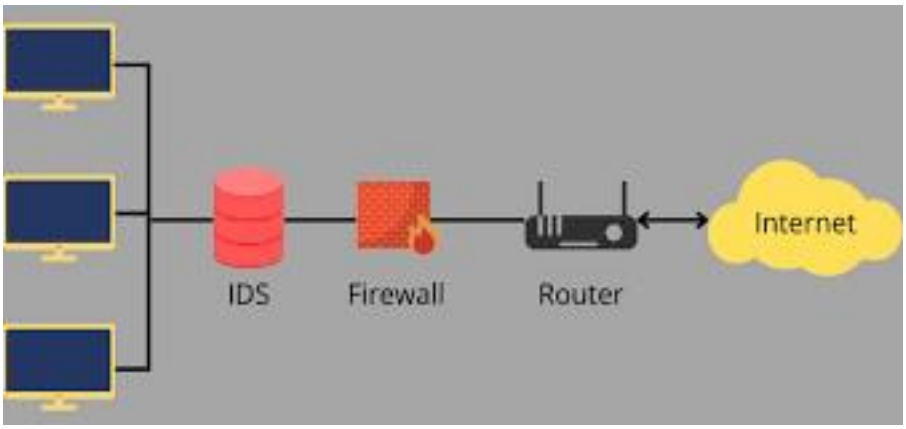

FIGURE 2.1 – IDS

2.3 Caractéristiques d'un système de détection d'intrusion

Pour obtenir un système de détection d'intrusions (IDS) performant, il est essentiel qu'il réponde à quatre impératifs majeurs. Voici une reformulation de ces caractéristiques clés :

1. **L'Auto-protection (Résilience) :** Le système doit être capable d'assurer sa propre intégrité. Cela signifie qu'il doit détecter toute tentative de sabotage ou de modification non autorisée pour l'arrêter. Un IDS incapable de se protéger lui-même devient vite le maillon faible de la sécurité.
2. **L'Efficience (Légèreté) :** Il ne faut pas que la surveillance perturbe l'outil de travail. L'IDS doit fonctionner de façon optimale, avec le moins de ressources possibles (processeur, mémoire, bande passante) afin de ne pas ralentir les performances globales du système qu'il protège.
3. **L'Adaptabilité (évolutivité) :** Un réseau est un milieu vivant. Le système doit pouvoir s'adapter dynamiquement aux évolutions techniques de l'infrastructure ainsi qu'aux changements d'habitudes des utilisateurs, afin d'éviter de devenir obsolète ou de générer trop de fausses alertes.
4. **La Flexibilité (Personnalisation) :** La sécurité n'est pas une solution unique. L'outil doit être très simple à configurer afin de s'adapter parfaitement à la stratégie de sécurité et aux règles propres à chaque organisation.

2.4 Différents types d'IDS

Les IDS peuvent être classés en trois types : N-IDS, H-IDS et IDS hybrides

1. **La détection d'intrusion basée sur l'hôte (H-IDS) :** Un HIDS est particulièrement adapté à la détection des menaces ciblant spécifiquement l'hôte, comme les tentatives d'accès non autorisé, l'exécution de logiciels malveillants ou l'usurpation de privilèges. L'un des principaux avantages des HIDS réside dans leur capacité à détecter les menaces

locales susceptibles d'échapper à la surveillance réseau. De plus, les HIDS sont particulièrement efficaces dans les environnements où la visibilité du réseau est limitée, comme sur les terminaux chiffrés ou dans les systèmes cloud.[7]

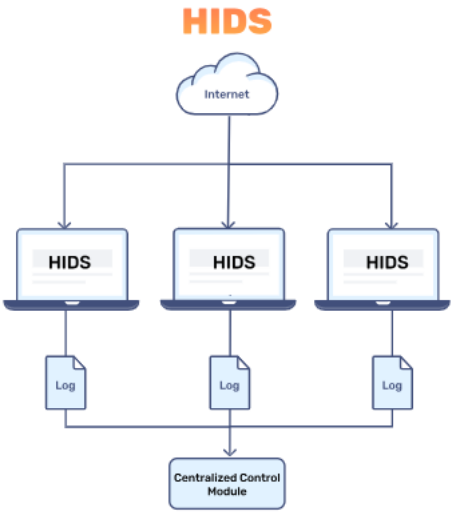

FIGURE 2.2 – Exemple H-IDS

2. **La détection d'intrusion réseau (N-IDS) :**Un NIDS utilise des méthodes telles que l'inspection approfondie des paquets (DPI) et l'analyse des flux pour identifier un large éventail de menaces, notamment les attaques par déni de service (DoS), la diffusion de logiciels malveillants et l'analyse des ports. L'un des avantages d'un NIDS est qu'il peut surveiller plusieurs périphériques à la fois, ce qui le rend adapté aux réseaux distribués à grande échelle. Cependant, les NIDS rencontrent des difficultés dans les environnements à fort volume de trafic ou contenant des données chiffrées. Les performances d'un système de détection d'intrusion réseau (NIDS) peuvent se dégrader en cas de charge de trafic excessive, entraînant des pertes. [7]

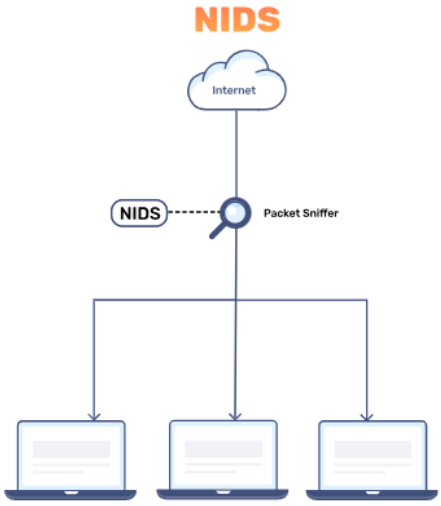

FIGURE 2.3 – Exemple N-IDS

3. **Système de détection d'intrusion Hybride :**Les systèmes de détection d'intrusion hybrides (IDS hybrides) combinent les capacités des IDS réseau (NIDS) et des IDS hôte (HIDS) pour offrir une solution de sécurité plus complète. Ces systèmes tirent parti des atouts des deux approches en surveillant à la fois le trafic réseau et l'activité des hôtes. En corrélant les données provenant de sources multiples, les IDS hybrides améliorent la précision de la détection des menaces et réduisent les faux positifs. Un avantage clé des IDS hybrides réside dans leur capacité à fournir une analyse contextuelle. Par exemple, une anomalie réseau détectée par le NIDS peut être corrélée à un accès suspect à un fichier sur un hôte, permettant ainsi une détection plus efficace des attaques en plusieurs étapes, telles que les APT. [7]

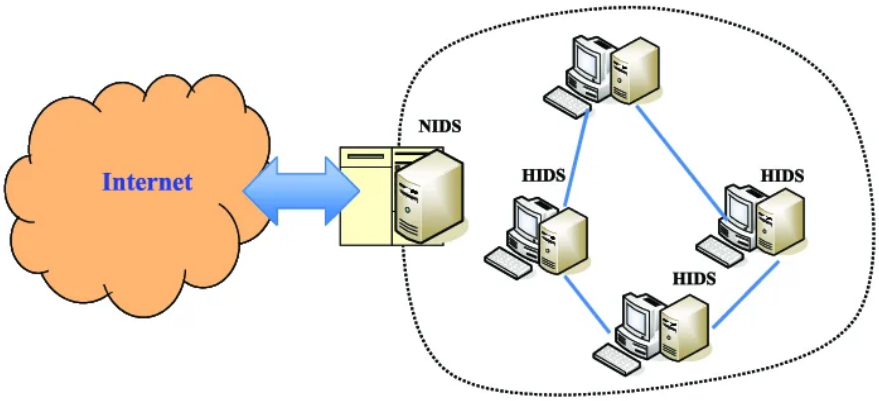

FIGURE 2.4 – Exemple IDS hybride

2.5 Architecture des IDS

Nous décrivons dans cette sous-section les trois composants qui constituent classiquement un système de détection d'intrusions. La figure illustre les interactions entre ces trois composants. [8]

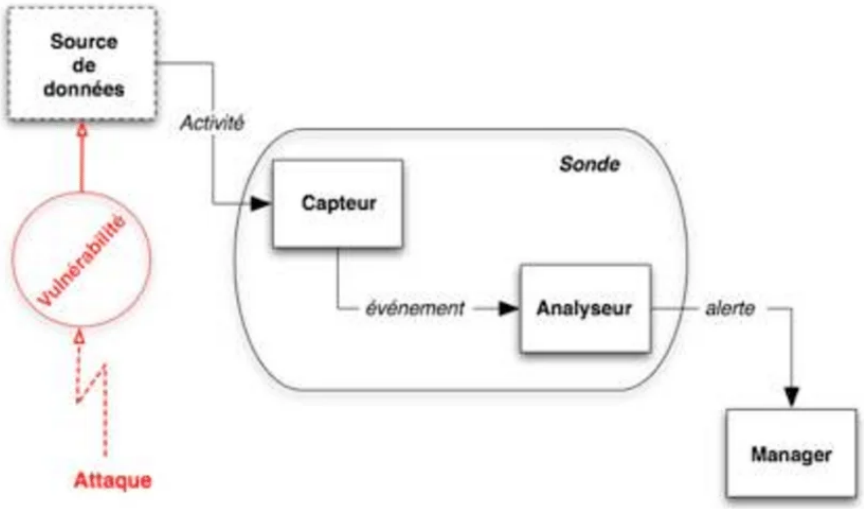

FIGURE 2.5 – Architecture classique d'un IDS

1. **Source de données :**Dispositif générant de l'information sur les activités des entités du système d'information. Il existe différents types de données provenant de plusieurs sources (réseau, système, application et alertes).
2. **Sonde :** Un ou plusieurs capteurs couplés avec un analyseur.
3. **Capteur :** Le capteur observe l'activité du système par le biais d'une source de données et fournit à l'analyseur une séquence d'évènements qui informe de l'évolution de l'état du système. Le capteur peut se contenter de transmettre directement ces données brutes, mais en général un prétraitement est effectué. [8] On distingue classiquement trois types de capteurs en fonction des sources de données utilisées pour observer l'activité du système :
   (a) **Les capteurs système :** Qui collectent des données produites par les systèmes d'exploitation des machines, notamment par le biais des journaux d'audit système ou par celui des appels système invoqués par les applications.
   (b) **Les capteurs réseau :** Qui collectent les données en écoutant le trafic réseau entre les machines par le biais d'une interface spécifique. On parle alors de NIDS
   (c) **Les capteurs applicatifs :** Qui collectent les données produites par une application particulière, avec laquelle des utilisateurs sont susceptibles d'interagir, comme un serveur web ou un serveur de base de données. [8]
4. **L'analyseur :** L'objectif de l'analyseur est de déterminer si le flux d'évènements fourni par le capteur contient des éléments caractéristiques d'une activité malveillante. [8]
5. **Alerte :** Un message formaté émis par l'analyseur s'il trouve des activités intrusives dans une source de données.
6. **Le Manager :** Le manager est responsable de la présentation des alertes a l'opérateur (fonction de console de management). Il peut également réaliser les fonctions de corrélation d'alertes, dans la mesure de leur disponibilité. Enfin, il peut assurer le traitement de l'incident, par exemple au travers des fonctions suivantes [8] :
   — confinement de l'attaque, qui a pour but de limiter les effets de l'attaque ;
   — éradication de l'attaque, qui tente d'arrêter l'attaque ;
   — recouvrement, qui est l'étape de restauration du système dans un état sain ;
   — diagnostic, qui est la phase d'identification du problème, de ses causes et qui peut éventuellement être suivi d'actions contre l'attaquant (fonction de réaction).[4]

2.6 Vocabulaire (notions) utilisé dans la détection d'intrusions

La détection d'intrusion possède un large champ de notions et de termes, que nous allons définir ci-dessous :

- **Attaque ou intrusions :** résulte de la pénétration non autorisée dans un système, elle constitue généralement la première phase d'une cyberattaque et peut être considérée comme un incident ou une anomalie délibérée.[5]
- **anomalie :** c'est une exception, elle peut induire un fonctionnement anormal du système et conduire à une violation de la politique de sécurité. Elle peut être à l'origine accidentelle ou volontaire.[5]
- **Système :** ensemble constitué d'un ou plusieurs capteurs, d'un ou plusieurs analyseurs et d'un ou plusieurs managers.
- **Détection d'intrusions :** est définie comme étant l'ensemble des pratiques et des mécanismes utilisés qui permettent de détecter des problèmes pouvant conduire à des violations de la politique de sécurité. [5]
- **Vulnérabilités :** ce sont des failles permettant aux attaquants d'accéder au réseau ou au système, dues à une mauvaise configuration ou à une implémentation incorrecte d'un système logiciel ou matériel.
- **Log (trace d'audit) :** c'est un fichier système à analyser.
- **Classification des attaques :** un IDS a une capacité de classifier une attaque.
- **La signature :** ce sont les règles ou les caractéristiques qui différencient les attaques, les virus, les vers, etc., des autres événements, permettant ainsi aux IDS d'identifier les attaques.
- **Administrateur :** c'est une personne chargée de la sécurité du système. C'est à lui de déployer l'IDS et de surveiller ses alertes.

2.7 Emplacement d'un système de détection d'intrusions

Le placement des IDS va dépendre de la politique de sécurité menée. Il existe plusieurs endroits stratégiques où il convient de placer un IDS tels que :

— Dans la zone démilitarisée (attaques contre les systèmes publics),

— Dans le (ou les) réseaux privée (intrusions vers ou depuis le réseaux interne),

— Sur la patte extérieure du firewall (détection de signes d'attaques parmi tout le trafic entrant et sortant, avant que n'importe quelle protection intervienne).

Il est important de bien définir les zones sensibles du système (réseau), ainsi que les zones les plus attractives pour un pirate. Le schéma suivant illustre les trois positions que peut y prendre un IDS dans un réseau local :

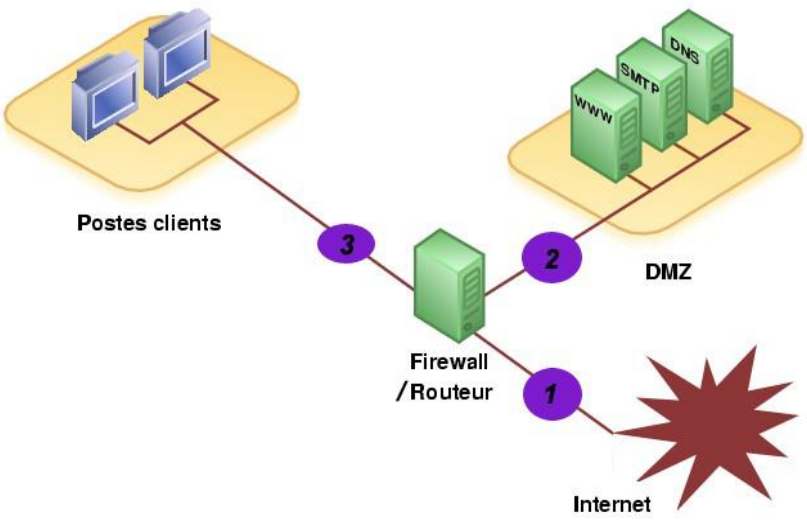

FIGURE 2.6 – Placement d'un IDS

— **Position (1) :**Positionné avant le pare-feu, l'IDS dispose d'une visibilité complète sur les attaques frontales en provenance du réseau extérieur et peut ainsi détecter toutes les tentatives d'intrusion avant même qu'elles ne soient bloquées.

— **Position (2) :**En plaçant l'IDS dans la DMZ, on concentre la surveillance sur les menaces qui ont franchi la première barrière du pare-feu. À cet endroit, le système ne signale plus les tentatives d'intrusion simples ou automatisées, mais détecte des attaques plus complexes et ciblées, reflétant un niveau de compétence supérieur de la part des attaquants.

— **Position (3) :**Placé sur le réseau local, l'IDS permet de surveiller les menaces internes, à l'origine de 80 des incidents de sécurité. Ce positionnement est stratégique pour détecter les activités malveillantes venant des employés ou repérer rapidement les chevaux de Troie introduits par une navigation imprudente, permettant ainsi de les éradiquer immédiatement

2.8 Classification des IDS

2.8.1 Sources et nature des données à analyser

Parmi les éléments les plus importants dans le fonctionnement des IDS figure la source des données sur laquelle l'IDS s'appuie pour initier son processus de sécurité. Ces données peuvent provenir de plusieurs sources, telles que les logs générés par le système d'exploitation ou les applications, ainsi que des alertes déjà signalées par d'autres IDS.

2.8.2 Méthodes de détection

Un IDS repose sur le principe que le comportement d'un intrus sera sensiblement différent de celui d'un utilisateur légitime et que de nombreuses actions non autorisées seront détectables.

Les systèmes de détection d'intrusion sont classés en trois catégories. Les différents types de techniques de détection d'intrusion sont énumérés ci-dessous :

- Systèmes de détection basés sur la signature
- Systèmes de détection d'intrusion basés sur les anomalies
- Systèmes de détection basés sur les spécifications.

2.8.2.1 Approche par signature

Les méthodes d'analyse des événements collectés basées sur des signatures d'intrusion consistent a les comparer a des scénarios d'attaques déjà connus. l'analyseur parcourt les données et les transforme en une d'action selon un modèle particulier . une fois la transformation accomplie, une étape de comparaison est alors effectuée pour identifier les événements connus par l'analyseur. Cette méthode est rapide est facile a implanter et de nombreux systèmes de détection d'intrusions l'utilisent. le nombre de fausses alarmes est relativement faible, voire inexistant. Son défaut majeur réside dans la nécessité de mettre à jour sa base de signatures régulièrement afin de faire face aux nouvelles attaques. Cette méthode est inefficace pour contrer des attaques dont la signature n'a pas été précédemment enregistrée dans le système de détection d'intrusion. suite elle présente des limites structurelles importantes :

2.8.2.2 Approche comportementale (détection d'anomalies)

Ces méthodes de détection d'intrusion sont basées sur la comparaison des événements collectés par rapport à des profils de comportements normaux associés à des utilisateurs ou à des applications. Comme la méthode précédente, les événements enregistrés sont convertis en une série d'actions particulières et comparés à des profils prédéfinis en se basant sur une méthode statique et les profils identifiés comme normaux. Le système évalue chaque comportement et parvient à distinguer les actions ou les traitements étranges qui se distinguent des profils enregistrés. Ses systèmes n'ont pas besoin de maintenance coûteuse et aucune connaissance antérieure des attaques n'est requise. Néanmoins, ce type de système génère un nombre assez important d'alarmes. La phase d'apprentissage des comportements est indispensable et doit être réalisée dans un environnement stable et non hostile. Cette méthode ne peut pas être mise en place dans un environnement dynamique où les systèmes, les utilisateurs et les applications changent fréquemment. Parmi les comportements anormaux typiques susceptibles d'être détectés, on peut citer :

- Utilisation abusive des protocoles réseau tels que le chevauchement de fragments IP et l'exécution d'un protocole standard sur un port furtif
- Des schémas de circulation inhabituels, tels qu'une augmentation de la comparaison des paquets UDP et TCP,
- Modèles suspects dans la charge utile de l'application.

2.8.2.3 Approche spécifications

Ce système de détection a pour but de surveiller les processus, en comparant les données réelles au comportement attendu du programme. Si une activité anormale est détectée, une alerte est automatiquement envoyée.
Pour rester efficace, ce système nécessite un entretien et une mise à jour réguliers, en particulier lorsqu'il y a changement des programmes surveillés. Cela lui permet de détecter autant des attaques connues que des attaques nouvelles tout en conservant un taux de faux positifs généralement inférieur à celui des systèmes reposant uniquement sur la détection d'anomalies.

2.8.3 Lieu de l'analyse des données

Les systèmes de détection d'intrusions peuvent être classés aussi en se basant sur la localisation réelle de l'analyse des données :

Paramètres	IDS distribué	IDS centralisé
Tolérance aux pannes	Élevée (absence de point unique de défaillance)	Faible (dépendance au serveur central)
Scalabilité	Élevée (ajout de nœuds possible et facile)	Limitée par la capacité du serveur
Configuration dynamique	Flexible (modification indépendante des nœuds)	Rigide (impact sur tout le système)
Charge (overhead)	Charge répartie (charge distribuée entre nœuds)	Forte charge (surcharge du serveur central)
Exécution	Plus complexe (coordination entre composants)	Plus simple (traitement centralisé unique)
Résistance aux attaques	Plus robuste grâce à la distribution	Plus vulnérable (point unique de défaillance)

TABLE 2.1 – Comparaison entre IDS centralisé et distribué

2.8.4 Fréquence de l'analyse

La fréquence d'utilisation d'un système de détection d'intrusion peut se présenter sous deux formes principales :

— **Surveillance périodique (IDS offline) :** examine périodiquement les données après leur collecte à la recherche d'une éventuelle attaque, intrusion ou anomalie passée, une faible consommation de ressources par contre la détection prend du temps ce qui peut entrainer des dégâts dangereux par la suite.

— **Surveillance en temps réel (IDS online) :** La detection se base sur une surveillance en temps réel en analysant continuellement les données afin d'augmenter le niveau de sécurité dans des contextes sensibles, résultat une détection rapide et une réaction immédiate face aux menaces, par contre ce type d'IDS peut entrainer une consommation importante des ressources, ce qu'il le rend non préférable en cas de ressources précieuses.

2.8.5 Comportement en cas de détection d'une attaque

On peut classer les IDS aussi par leurs réactions lorsqu'une attaque ou menace est détectée, il existe deux types de réponses. La réponse passive est disponible dans tous les IDs, l'autre active est moins implémentée.

— **Réponse passive :**La réponse passive d'un IDS consiste à enregistrer les intrusions détectées dans un fichier de log qui sera analysé par le responsable de sécurité ou générer des alarmes, envoi d'un E-mail à un ou plusieurs utilisateurs, etc. Ceci permet de remédier aux failles de sécurité pour empêcher les attaques enregistrées de se reproduire, mais elle n'empêche pas directement une attaque de se produire.

— **Réponse active :** la réponse active vise, contrairement à l'aspect passif, à stopper l'attaque sans intervention humaine. L'une des méthodes utilisées est de reconfigurer les

pare-feu, permettant de bloquer le trafic malveillant, ainsi que l'interruption des sessions TCP afin d'empêcher toute action de l'attaquant.

2.9 Mesures performance des IDS :

La mesure de la performance des IDS (Intrusion Detection Systems) est essentielle pour évaluer leur efficacité dans la détection des intrusions dans un réseau.

1. **Matrice de confusion**La matrice est une matrice carrée de taille N × N, où N représente le nombre de classes de sortie. Chaque ligne de la matrice représente le nombre d'instances d'une classe prédite et chaque colonne représente le nombre d'instances de la classe réelle. Ceci fournit une ventilation, classe par classe, du nombre de prédictions exactes et inexactes effectuées par un classificateur pour les tâches de classification. La catégorisation peut être binaire ou multiclasse. La matrice de confusion révèle les performances du classificateur, ses classifications correctes et les nombreux types d'erreurs qu'il peut commettre.[9]

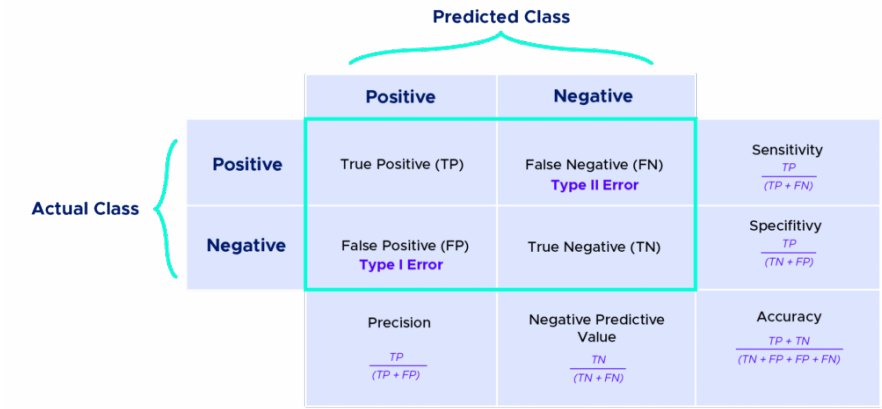

FIGURE 2.7 – Matrice de Confusion

- **Vrai positive (TP) :** Les valeurs réelles et la prédiction sont toutes deux positives.
- **Vrai négative (TN) :** La valeur réelle est négative et la prédiction est négative.
- **Faux positive (FP) :** Bien que la prédiction soit positive, la valeur réelle est négative. On parle alors d'« erreur de type I ».
- **Faux négatif (FN) :** Alors que l'échantillon est prédit comme négatif, il est en réalité positif. On parle également d'« erreur de type II ».

À partir de cette matrice, plusieurs métriques de performance peuvent être calculées pour évaluer la qualité du modèle de classification, telles que :

— **Exactitude (Accuracy) :** La précision est la fraction de prédictions correctes parmi toutes les prédictions ou la fréquence à laquelle une prédiction est correcte.[9]

Accuracy = (Nombre d'instances correctement classées) / (Nombre total d'instances)

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN}$$

La précision est particulièrement utile lorsque les classes du jeu de données sont équilibrées, c'est-à-dire lorsqu'elles comportent un nombre d'instances approximativement égal.

— **Précision :** S'il y a plus de deux classes de sortie, la précision de la classification ne renseigne pas sur les classes correctement prédites. Dans ce cas, la justesse est une mesure plus appropriée. La justesse correspond à la proportion de résultats positifs correctement prédits.

$$Prcision = \frac{TP}{TP + FP}$$

La précision n'évalue pas correctement les performances face aux cas négatifs. Elle ne tient pas compte non plus des faux négatifs (erreurs de type II), qui correspondent à des cas réellement positifs mais incorrectement classés comme négatifs.

— **Rappel (Recall) :** Ce paramètre mesure la proportion de cas positifs réels correctement prédits, ou la précision avec laquelle le modèle prédit les cas positifs. Il est calculé comme suit :

$$Rappel = \frac{VP}{VP + FN}$$

Le rappel est particulièrement précieux lorsqu'il est nécessaire de minimiser les faux négatifs (erreurs de type II).

— **F1 - Score :** Le score F1 est la moyenne harmonique de précision et de rappel, calculée comme suit :

$$F1 - Score = 2 \times \frac{Precision \times recall}{Precision + recall}$$

Le score F1 est une mesure appropriée lorsque la précision et le rappel sont tout aussi importants. Il équilibre les deux mesures et fournit une seule mesure du rendement. Le score F1 est couramment utilisé dans les tâches de récupération d'informations.

— **Taux de faux positifs :** Le taux de faux positifs (FPR) identifie la proportion de résultats négatifs classés comme positifs :

$$FPR = \frac{FP}{FP + TN}$$

— **Taux de mauvaise classification :** également appelé « taux d'erreur », il indique

à quel point le modèle est erroné.

$$tauxdemauvaiseclassification = \frac{FP + FN}{FP + FN + TP + TN}$$

2.10 Limites et vulnérabilités des IDS

Les IDS comme tout autre système informatique, ont leurs limites malgré leurs avantages, quelle que soit l'architecture N-IDS ou H-IDS

- Faux positifs (très fréquent) : L'IDS détecte une attaque... alors qu'il n'y en a pas
- Faux négatifs (très dangereux) : L'IDS ne détecte pas une vraie attaque
- Difficulté avec les attaques zero-day : attaques nouvelles, inconnues
- Difficulté avec le trafic chiffré : HTTPS, VPN, etc.
- Pollution des IDS :
  - **Consommation de ressources :** Une consommation élevée due à l'analyse en continu du réseau, ainsi qu'au traitement d'un grand volume de données et au stockage des logs.
  - **Perte de paquets :** La vitesse de transmission peut parfois dépasser la vitesse d'écriture et de traitement, ce qui cause des pertes.
  - **Vulnérabilités au déni de service :** L'attaquant peut tenter de mettre l'IDS hors service en provoquant un déni de service sur celui-ci ou sur le système d'exploitation sur lequel il fonctionne. Une fois cela réalisé, il peut agir librement au sein du système.
- Techniques d'évasion (attaque intelligente) : les attaquants contournent l'IDS
- Manque de réaction (IDS =! IPS) : IDS détecte mais n'agit pas

2.11 Les systèmes de prévention d'intrusion (IPS)

Un IPS est un système de sécurité qui permet non seulement de détecter les activités suspectes, mais aussi d'y répondre en appliquant des mécanismes de prévention afin de limiter les risques potentiels. Il analyse le trafic réseau en le comparant à un comportement normal, puis bloque automatiquement toute activité jugée malveillante avant qu'elle ne cause des dommages au système.

2.12 Similarités et différences entre IDS et IPS

2.12.1 Similarités

Les premiers processus des IDS et des IPS sont similaires. Les deux détectent et surveillent le système ou le réseau à la recherche d'activités malveillantes. Voyons leurs points communs :

— **Surveiller :** Les IDS et les IPS, une fois installés, sont chargés de surveiller un réseau ou un système en fonction de paramètres définis selon les besoins de l'utilisateur et les caractéristiques de son infrastructure réseau. Ils disposent ainsi de la capacité d'analyser l'ensemble du trafic entrant et sortant.

— **Détection des menaces :** Ils utilisent une procédure de détection similaire, consistant à analyser l'ensemble des données circulant dans le réseau, puis à les comparer à une base existante contenant les menaces connues. En cas de similarité suspecte, le paquet est immédiatement signalé comme une menace.

— **Apprendre :** Les IDS et les IPS utilisent des méthodes d'apprentissage similaires, en s'appuyant sur des technologies modernes telles que l'apprentissage automatique, afin de s'entraîner sur les menaces existantes et de parvenir à distinguer les attaques du trafic normal.

— **Alerte :** Les deux systèmes envoient des alertes au personnel de sécurité lors de la détection d'une menace, afin de les informer et de leur permettre de prendre les mesures nécessaires avant qu'il ne soit trop tard.

2.12.2 Différences

La différence principale entre IDS et IPS est que IDS fonctionne comme un système de surveillance et de détection, tandis que IPS fonctionne comme un système de prévention en plus de la surveillance et de la détection. Certaines différences sont :

— **Reponse :** Pour les IDS, ils se contentent de signaler et d'alerter le personnel de sécurité, et c'est à ce dernier de prendre les mesures nécessaires afin de contrer la menace et d'éviter d'éventuels dégâts. En revanche, les IPS sont des systèmes qui, en plus de surveiller, détecter et alerter, permettent également d'empêcher automatiquement l'attaque.

— **placement :** Les IDS sont placés généralement à la périphérie du réseau, ce qui leur permet d'avoir une visibilité maximale sur les données afin de collecter tous les événements, les enregistrer et détecter les violations. Par contre, les IPS sont déployés juste derrière le pare-feu, avec la possibilité d'interagir avec le trafic entrant et sortant afin de mieux prévenir les intrusions.

— **Protection :** La sécurité proposée par les IDS n'est pas totalement efficace et dépend fortement de la rapidité de réaction du personnel pour nettoyer le système ou le réseau dans les plus brefs délais. En revanche, les IPS permettent d'assurer une prévention automatique.

2.13 Exemples des IDS existants :

Il existe aujourd'hui plusieurs études conçues pour améliorer les systèmes de détection d'intrusion (IDS), chacune conçu pour répondre à des besoins spécifiques en matière de sécurité. Certains sont spécialisés dans la détection de types particuliers d'attaques, tandis que d'autres adoptent une approche plus générale afin de couvrir un large éventail de menaces. Par ailleurs, ces systèmes évoluent constamment, allant des solutions plus anciennes aux nouvelles générations d'IDS intégrant des techniques avancées. Dans ce qui suit, nous présentons quelques exemples de ces études et en présentons un peu les résultats finaux.

2.13.1 Survey on Federated Learning for Intrusion Detection System : Concept, Architectures, Aggregation Strategies, Challenges, and Future Directions [1]

1. **Auteurs :** ANSAM KHRAISAT,AMMAR ALAZAB,SARABJOT SINGH,TONY JAN et ALFREDO JR. GOMEZ.
2. **Source et Date :** Publié dans ACM Computing Surveys, Volume 57, en octobre 2024
3. **Objectif de la recherche :** L'objectif de l'étude est de fournir un exemple de l'application de l'apprentissage fédéré dans les systèmes de détection d'intrusions (IDS) afin

de résoudre les problèmes de confidentialité des données et de scalabilité des approches centralisées traditionnelles.

4. **Stratégies d'Agrégation analysées :** L'article détaille plusieurs méthodes pour combiner les mises à jour des modèles locaux :
   - **FedAvg (Federated Averaging) :** La méthode de base par calcul de moyenne pondérée
   - **WFedAvg :** Variante tenant compte de l'hétérogénéité des données.
   - **Méthodes robustes :** Utilisation de la médiane (MFedAvg), du Trimmed Mean ou de l'algorithme Krum pour contrer les mises à jour malveillantes ou aberrantes
5. **Défis identifiés :**Les auteurs listent les obstacles majeurs tels que les attaques par empoisonnement de données, l'hétérogénéité du matériel, la distribution non-IID des données (non uniformes) et le coût élevé des communications.

2.13.2 Toward Enhancing Privacy Preservation of a Federated Learning CNN Intrusion Detection System in IoT : Method and Empirical Study [2]

1. **Auteurs :** DAMIANO TORRE,ANITHA CHENNAMANENI,JAEYUN JO, GITIKA VYAS et BRANDON SABRSULA,
2. **Source et Date :** Publié dans IEEE Transactions on Industrial Informatics en 2021
3. **Objectif de la recherche :**Développer une approche d'apprentissage profond fédéré permettant de détecter et de limiter les cybermenaces dans les systèmes cyber-physiques industriels (CPS), tout en assurant la confidentialité des données échangées.
4. **Objectif de la recherche :** Développer un système de détection d'intrusion (IDS) reposant sur l'apprentissage fédéré et des réseaux de neurones convolutionnels unidimensionnels (CNN 1D), intégrant plusieurs mécanismes de préservation de la confidentialité afin de renforcer la sécurité des réseaux IoT.
5. **Évaluation empirique :** Tests sur sept jeux de données IoT récents (TON-IoT, IoT-23, BoT-IoT, CIC IoT 2023, etc.) totalisant 415 millions d'enregistrements
6. **Défis identifiés :**
   - Vulnérabilité aux attaques sophistiquées
   - Surcharge de calcul (Overhead)
   - Difficulté de distinction entre classes

2.13.3 Federated Learning for IoT Intrusion Detection [3]

1. **Auteurs :**Riccardo Lazzarini,Huaglory Tianfield et Vassilis Charissis.

2. **Source et Date :** Publié dans la revue AI (MDPI), Volume 4, en juillet 2023
3. **Objectif de la recherche :** L'étude évalue l'utilisation de l'apprentissage fédéré (FL) comme méthode pour implémenter la détection d'intrusion dans les environnements IoT. L'objectif est de proposer une alternative aux approches centralisées pour résoudre les problèmes de confidentialité des données des utilisateurs et les goulots d'étranglement liés au transfert de gros volumes de données vers un serveur central.
4. **Concepts clés et Taxonomie :**
   - **Architecture :** Le système utilise un partitionnement horizontal des données, où chaque client possède des échantillons différents mais partage le même espace de caractéristiques (mêmes colonnes de données).
   - **Déploiement :** L'infrastructure est composée de quatre clients distribués et d'un serveur agissant comme agrégateur. Le framework utilisé pour l'implémentation est Flower.
   - **Modèle partagé :** Un réseau de neurones artificiels (ANN) "shallow" (superficiel) composé de trois couches : une couche d'entrée (24 neurones), une couche cachée (16 neurones) et une couche de sortie adaptée au type de classification.
5. **Stratégies d'Agrégation analysées :**
   - **FedAvg (Federated Averaging) :** Utilisé comme algorithme de référence pour agréger les poids des modèles locaux.
   - **Comparaisons :** L'étude évalue également trois autres algorithmes d'agrégation : FedAvgM (avec Momentum), FedAdam et FedAdagrad.
6. **Défis et Limites identifiés :**
   — **Convergence et hétérogénéité :** Les auteurs soulignent que FedAvg peut avoir des difficultés à converger lorsque les données sont hautement hétérogènes ou non identiquement distribuées (non-IID).
   — **Performance décevante des algorithmes adaptatifs :** Dans le scénario testé (nombre limité de clients), les algorithmes FedAdam et FedAdagrad ont montré des performances inférieures et parfois peu fiables par rapport à FedAvg.
   — **Simplicité du modèle :** L'utilisation d'un modèle ANN superficiel est une limite reconnue.

2.13.4 An optimal federated learningbased intrusion detection for IoT environment [4]

1. **Auteurs :** A. Karunamurthy,K. Vijayan,Pravin R. Kshirsagar et Kuan Tak Tan.
2. **Source et Date :** Publié dans Scientific Reports (Nature Portfolio) le 13 mars 2025

3. **Objectif de la recherche :** L'étude propose une approche de détection d'intrusion basée sur l'apprentissage fédéré (FL) pour les réseaux IoT afin de surmonter la complexité des nouveaux modèles d'attaque tout en garantissant la confidentialité et la sécurité des données décentralisées.
4. **Concepts clés et Taxonomie :**
   - **Architecture décentralisée :** Le système utilise des instances IoT distribuées connectées via des passerelles (gateways) qui hébergent le module de détection, réduisant ainsi la latence et la dépendance à un serveur central.
   - **Sélection de caractéristiques (Optimization) :** la touche innovante dans cette étude est l'utilisation de l'algorithme d'optimisation Chimp pour sélectionner les caractéristiques optimales du trafic réseau, réduisant ainsi la dimensionnalité (de 34 à 17 caractéristiques essentielles) et la complexité de calcul.
   - **Classifieur :** Utilisation de réseaux de neurones convolutionnels (CNN) pour l'entraînement local et la classification des attaques.
5. **Stratégie d'Agrégation :**
   - **FL Average (Moyenne Fédérée) :** Le serveur central agrège périodiquement les paramètres des modèles CNN locaux pour créer un modèle global "intelligent" qui est ensuite redistribué aux instances IoT.
6. **Défis et Limites identifiés :**
   - **Vulnérabilité à l'empoisonnement :** Si un modèle local est affecté par des données malveillantes et envoie des paramètres erronés, cela peut dégrader la précision du modèle global.
   - **Complexité de calcul :** L'intégration de l'algorithme d'optimisation entraîne une légère augmentation de la charge de calcul, ce qui peut représenter une contrainte pour les appareils IoT disposant de ressources limitées, notamment les capteurs à basse consommation.

2.14 Conclusion

Les systèmes de détection d'intrusion sont des solutions de plus en plus fiables pour la sécurité des systèmes et des réseaux, grâce aux avantages qu'ils présentent par rapport aux autres outils. Cependant, comme nous l'avons vu dans ce chapitre, les IDS possèdent également des limites et des inconvénients, ce qui a poussé à améliorer ces systèmes en intégrant l'intelligence artificielle afin de réduire ces limites et d'améliorer la sécurité proposée.
C'est ce que nous allons découvrir dans les prochains chapitres, en essayant d'intégrer quelques technologies de l'intelligence artificielle, d'analyser les résultats, puis de proposer une solution optimale.

Chapitre 3

Solution proposée

3.1 Introduction

3.2 Introduction à l'intelligence artificielle :

L'IA fait référence à un ensemble de méthodes et de technologies visant à faire fonctionner intelligemment des ordinateurs ou d'autres dispositifs. L'IA consiste essentiellement en un ensemble d'algorithmes fonctionnant sur des données (généralement volumineuses).Les applications de l'apprentissage machine ont eu récemment un impact inattendu dans de nombreux domaines de la science et de la technologie. Il existe un large consensus sur la progression régulière de la recherche sur l'IA et sur l'augmentation probable de son impact sur le futur de la société. [10]

— L'intelligence artificielle regroupe plusieurs sous-domaines qu'on va voir juste après, dont on peut citer l'apprentissage automatique, l'apprentissage profond et l'apprentissage fédéré.

3.2.1 Apprentissage automatique

Le domaine de l'intelligence artificielle a pour objectif de parvenir à simuler l'intelligence humaine et en particulier l'apprentissage de nombreuses tâches. Deux méthodes sont alors possibles pour apprendre :

- L'apprentissage par coeur consiste à mémoriser explicitement tous les exemples possibles afin de pouvoir les restituer
- L'apprentissage par généralisation a pour objectif d'extraire des règles implicites à partir d'une quantité d'exemples afin de les réappliquer à de nouvelles situations jamais rencontrées. L'apprentissage par coeur est relativement aisé pour une machine à condition de disposer des exemples. En revanche, l'apprentissage par généralisation est difficile car il demande d'extraire des règles qui ne sont pas explicitement mentionnées dans les exemples. Ce défi constitue le coeur l'apprentissage automatique.

Comme expliqué précédemment, l'apprentissage automatique est un champ de l'IA porté sur l'analyse statistique de données d'apprentissage. Historiquement, cette branche est définie comme le développement de machines capables d'apprendre sans avoir été explicitement programmées à apprendre une tâche. [11]

3.2.1.1 Types d'apprentissage (Types of Learning)

1. **Apprentissage supervisé :**On appelle apprentissage supervisé la branche du machine learning qui s'intéresse aux problèmes pouvant être formalisés de la façon suivante : étant données n observations $\{\mathbf{x}_i\}_{i=1,\dots,n}$ décrites dans un espace X, et leurs étiquettes $\{y_i\}_{i=1,\dots,n}$ décrites dans un espace Y, on suppose que les étiquettes peuvent être obtenues

à partir des observations grâce à une fonction $\varphi : X \to Y$ fixe et inconnue :

$$y_i = \varphi(\mathbf{x}_i) + \varepsilon_i,$$

où ε_i est un bruit aléatoire. Il s'agit alors d'utiliser les données pour déterminer une fonction $f : X \to Y$ telle que, pour tout couple $(\mathbf{x}, \varphi(\mathbf{x})) \in X \times Y$, on ait :

$$f(\mathbf{x}) \approx \varphi(\mathbf{x}).$$

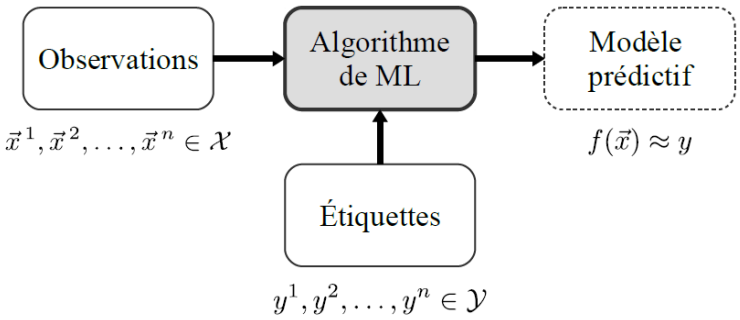

FIGURE 3.1 – Apprentissage supervisé

2. **Apprentissage non supervisé :** On appelle apprentissage non supervisé la branche du machine learning qui s'intéresse aux problèmes pouvant être formalisés de la façon suivante : étant données n observations $\{\mathbf{x}_i\}_{i=1,\ldots,n}$ décrites dans un espace X, il s'agit d'apprendre une fonction sur X qui vérifie certaines propriétés.

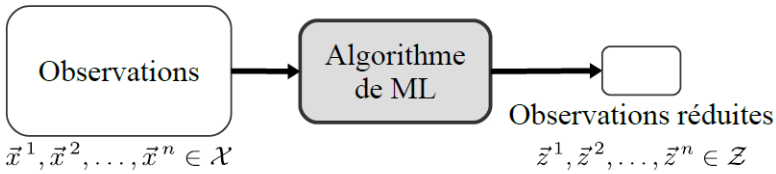

FIGURE 3.2 – Apprentissage non supervisé

3. **Apprentissage semi-supervisé :** comme on peut s'en douter, l'apprentissage semi-supervisé consiste à apprendre des étiquettes à partir d'un jeu de données partiellement étiqueté. Il est facile d'obtenir une banque de données contenant des centaines de milliers d'images, mais avoir pour chacune d'entre elles l'étiquette qui nous intéresse peut requérir énormément de travail.

4. **Apprentissage par renforcement :** Dans le cadre de l'apprentissage par renforcement, le système d'apprentissage peut interagir avec son environnement et accomplir des actions. En retour de ces actions, il obtient une récompense, qui peut être positive si l'action était un bon choix, ou négative dans le cas contraire. La récompense peut parfois venir après une longue suite d'actions ; c'est le cas par exemple pour un système apprenant à jouer au go ou aux échecs. Ainsi, l'apprentissage consiste dans ce cas à définir une politique, c'est-à-dire une stratégie permettant d'obtenir systématiquement la meilleure récompense possible.
   Les applications principales de l'apprentissage par renforcement se trouvent dans les jeux (échecs, go, etc) et la robotique.

3.2.2 Apprentissage en profondeur(Deep learning)

L'apprentissage profond est une sous-discipline du machine learning basée sur des réseaux de neurones artificiels composés de multiples couches, inspirés du fonctionnement du cerveau humain. Ces modèles permettent d'apprendre automatiquement des représentations complexes des données grâce à l'ajustement des poids et des biais entre les neurones, ce qui améliore progressivement la précision des prédictions.

Contrairement aux méthodes traditionnelles reposant sur des fonctions mathématiques explicites, le deep learning utilise des architectures profondes capables d'approximer des relations très complexes entre les données. Cette capacité lui confère une grande flexibilité et en fait une technologie centrale dans de nombreux domaines, tels que la vision par ordinateur, la détection d'intrusions ou encore les systèmes autonomes.

3.2.2.1 Réseau de neurones profond DNN

Les réseaux de neurones artificiels sont constitués de couches de neurones interconnectés, où chaque neurone applique une fonction mathématique appelée fonction d'activation. Les neurones d'une couche sont généralement connectés à ceux de la couche suivante, formant une structure en cascade où la sortie de chaque neurone devient une entrée pour la couche suivante.

L'utilisation de fonctions d'activation non linéaires est essentielle, car elle permet au réseau de modéliser des relations complexes, contrairement aux modèles linéaires. Bien que certaines architectures spécialisées comme les réseaux convolutifs introduisent des variations, tous les réseaux de neurones reposent sur ce principe fondamental.

Enfin, des éléments tels que le nombre de couches, le nombre de neurones par couche et le choix des fonctions d'activation constituent des hyperparamètres définis avant l'entraînement du modèle.

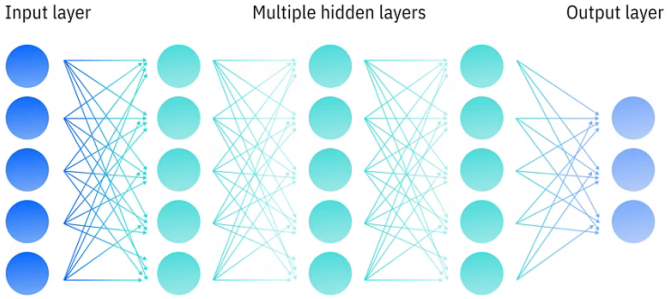

FIGURE 3.3 – Deep Neural Network Diagram

3.2.2.2 Convolutional neural networks (CNNs)

Les réseaux de neurones convolutionnels (CNN) sont des modèles particulièrement adaptés aux tâches de vision par ordinateur, car ils permettent de détecter efficacement des motifs locaux dans les données, comme les relations entre pixels d'une image. Contrairement aux réseaux entièrement connectés, les CNN réduisent le nombre de paramètres en utilisant des filtres (ou noyaux) qui parcourent l'image afin d'extraire des caractéristiques pertinentes.

L'opération principale, appelée convolution, consiste à appliquer ces filtres sur des portions de l'image pour détecter des motifs spécifiques (formes, contours, etc.). Cette approche permet de limiter la complexité du modèle, d'améliorer la généralisation et de réduire le risque de surapprentissage.

Au fil des couches, les CNN apprennent des caractéristiques de plus en plus complexes, formant des cartes de caractéristiques (feature maps). Ces informations sont ensuite utilisées par des couches finales, souvent entièrement connectées, pour réaliser des tâches comme la classification ou la détection.

3.2.2.3 Les réseaux de neurones récurrents (RNN)

Les réseaux de neurones récurrents (RNN) sont conçus pour traiter des données séquentielles, telles que les séries temporelles, la parole ou le langage naturel. Contrairement aux réseaux classiques, ils prennent en compte l'ordre des données en utilisant une boucle récurrente qui permet de conserver une mémoire interne, appelée état caché, contenant les informations des étapes précédentes.

Cette capacité permet aux RNN de modéliser des dépendances temporelles et contextuelles. Cependant, leur entraînement présente des difficultés, notamment les problèmes de gradient qui disparaît ou explose, limitant leur efficacité sur de longues séquences.

Pour surmonter ces limites, des variantes comme les réseaux LSTM et GRU ont été déve-

loppées, améliorant la gestion des dépendances à long terme et la stabilité de l'apprentissage.

3.2.3 Détection d'anomalies :

Cette approche repose sur la modélisation d'un «comportement normal» du système ou de l'entité surveillée, appelé profil, et la détection des écarts par rapport à ce profil
La détection d'anomalies peut servir à l'amélioration de la qualité des données par suppression ou remplacement des données anormales. L'exploitation de données sans anomalies permet d'aboutir à des résultats et des décisions plus sûres pour plusieurs applications.[12]

3.2.3.1 État de l'art :

La détection d'anomalies est un sujet qui intéresse beaucoup de chercheurs et qui a fait l'objet de nombreux travaux. Plusieurs méthodes ont été proposées pour la détection d'anomalies et chaque méthode a ses forces et ses faiblesses.Patcha et Park (2007) ont fait une revue des méthodes utilisées pour la détection d'intrusion. Une revue plus générale des techniques existantes couvrant plusieurs approches est proposée dans Aggarwal (2017) et Chandola et al.ANSAM KHRAISAT,AMMAR ALAZAB,SARABJOT SINGH,TONY JAN et ALFREDO JR. GOMEZ ont fait une revue en ce qui concerne l'utilisation de l'apprentissage fédérée dans les détections d'intrusions(2024). DAMIANO TORRE,ANITHA CHENNAMANENI, JAEYUN JO, GITIKA VYAS et BRANDON SABRSULA ont proposé une solution basée sur le Federated Learning et CNN (2025).Riccardo Lazzarini

1. **Jeux de données :**

3.3 Modèle proposé : AutoEncodeur + MLP + FedDistillation :

3.3.1 Autoencodeur

Un autoencodeur est un type de réseau de neurones non supervisé, conçu pour encoder des données d'entrée puis les reconstruire à partir de cette version compressée, appelée espace latent. Ce dernier est composé d'un ensemble de variables latentes, qui représentent les facteurs ou les règles expliquant la structure des données.
Le but est de reconstruire les données originales de la manière la plus fidèle possible.

3.3.1.1 Architecture generale :

L'architecture de base d'un autoencodeur se compose de trois éléments principaux :

— **l'encodeur** L'encodeur est une succession de couches affines et de fonctions d'activation réalisant une réduction de dimensionnalité. En "écrasant" progressivement le volume des données, il effectue une projection mathématique de l'espace d'entrée vers un espace de dimension inférieure, capturant les corrélations les plus robustes.

— **le bottleneck (ou espace latent)** La partie bottleneck, qui est la plus petite couche du réseau, est un espace à faible dimension qui capture les caractéristiques essentielles des données d'entrée.

— **le décodeur.** Le décodeur opère la transformation inverse, restaurant les données d'entrée. Son rôle est essentiel durant l'entraînement pour valider la qualité du code. Pour les modèles génératifs, il reste le composant principal capable de transformer un vecteur latent en une donnée intelligible (image, son, texte).
L'efficacité se calcule par l'erreur de reconstruction, souvent calculée via l'erreur quadratique moyenne (MSE). Cet écart guide l'optimisation des poids via la rétropropagation.

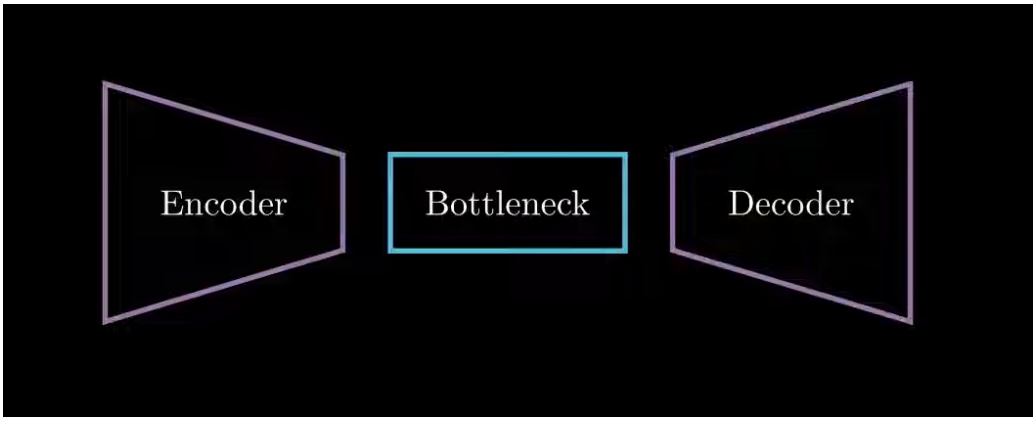

FIGURE 3.4 – Architecture de l'autoencodeur

Avantage Comparatif (vs PCA) : Là où l'Analyse en Composantes Principales (PCA) est limitée à des transformations linéaires, l'autoencodeur utilise des fonctions d'activation non-linéaires (ex : Sigmoïde ou ReLU). Cela lui permet de modéliser des variétés de données complexes que les méthodes statistiques traditionnelles ne peuvent pas appréhender.

3.3.1.2 Les Hyperparamètres

Le paramétrage d'un autoencodeur est une quête d'équilibre contre le surapprentissage (overfitting). Si le modèle dispose d'une capacité trop élevée, il risque de mémoriser les données plutôt que d'en comprendre la logique structurelle.

— **Taille du Code (Code Size) :** Dimension du bottleneck. Une taille trop faible entraîne une perte d'information cruciale (underfitting) ; une taille trop grande permet au réseau de copier l'entrée sans compression.

— **Profondeur du Réseau :** Le nombre de couches définit le niveau d'abstraction. Plus le réseau est profond, plus il peut capturer des caractéristiques complexes, au risque d'augmenter le coût computationnel.

— **Nombre de Nœuds par Couche :** Suit typiquement une architecture en "sablier". Cependant, dans les architectures spécialisées comme les autoencodeurs clairsemés (Sparse), les couches cachées peuvent être plus larges que l'entrée.

— **Fonction de Perte (Loss Function) :** C'est le moteur de l'optimisation par descente de gradient. Elle doit être choisie selon la nature des données (ex : Binary Cross-Entropy pour des entrées normalisées).

Tout ces Hyperparamètres et ces réglages ont donné naissance à une taxonomie diversifiée, chaque variante répondant à une contrainte mathématique spécifique.

3.3.1.3 Variantes des Autoencodeurs

1. **Autoencodeurs Sous-complets (Undercomplete)** Les autoencodeurs undercomplete sont des modèles de réduction de dimension où la couche cachée contient moins de neurones que l'entrée. Ce "goulot d'étranglement" force le réseau à compresser les données et à ne garder que les caractéristiques essentielles, évitant ainsi d'apprendre simplement à recopier les données (fonction identité). Cependant, si le modèle est trop puissant, il peut quand même contourner cette contrainte, ce qui rend ces autoencodeurs limités et peu flexibles.
2. **Autoencodeurs Régularisés(Regularized autoencoders)** Les autoencodeurs régularisés corrigent les limites des autoencodeurs undercomplete en introduisant une régularisation : des techniques qui contraignent ou modifient la manière dont le modèle calcule l'erreur de reconstruction. Ces termes de régularisation permettent non seulement de réduire le surapprentissage, mais aussi d'amener l'autoencodeur à apprendre des caractéristiques ou des fonctions utiles.
   — Sparse Autoencoders (SAE) :
   Ils imposent une contrainte de parcimonie. On autorise des couches larges, mais on pénalise l'activation simultanée de trop de neurones. La logique est qu'il n'est "profitable pour le réseau" d'activer un neurone que si la caractéristique détectée est hautement pertinente. On utilise pour cela la divergence de Kullback-Leibler (KL) pour mesurer l'écart entre la distribution d'activation réelle et une cible de parcimonie proche de zéro.
   — Contractive autoencoders (CAE) :
   Conçus pour être insensibles au bruit, ils utilisent une approche quasi-adversaire. On ajoute un terme de régularisation basé sur la norme de Frobenius de la matrice Jacobienne des activations de l'encodeur. Ce terme pénalise les variations de sortie face aux petites variations d'entrée
   — Denoising autoencoders (DAE) :
   Le modèle reçoit une entrée corrompue par un bruit gaussien mais doit reconstruire la version "propre". Cela force le réseau à découvrir les corrélations stables et robustes du signal.
3. **Autoencodeurs Variationnels (VAE)** Les autoencodeurs variationnels (VAE) sont des modèles génératifs capables d'apprendre une représentation compressée des données sous

forme de distributions de probabilité, afin de générer de nouvelles données similaires. Contrairement aux autoencodeurs classiques qui utilisent un espace latent discret, les VAE modélisent un espace latent continu à l'aide de deux vecteurs : la moyenne (μ) et l'écart-type (σ).

Cette approche probabiliste permet de générer de nouvelles données par interpolation et échantillonnage aléatoire, ce qui rend les VAE particulièrement puissants pour la génération. Leur fonction de perte combine l'erreur de reconstruction avec une régularisation (divergence KL) afin d'éviter le surapprentissage et de garantir des générations variées mais cohérentes.

Enfin, grâce à une technique appelée reparameterization trick, les VAE peuvent être entraînés efficacement. Une fois entraînés, ils peuvent générer de nouveaux exemples ressemblant aux données d'origine, avec la possibilité de contrôler les sorties dans des variantes avancées comme les VAE conditionnels.

3.3.2 MLP :

3.3.2.1 Definition au MLP :

l'algorithme d'apprentissage MLP le plus connu est la rétropropagation, qui est une généralisation de la règle des moindres carrés moyens (Du et Swamy 2014). Les poids peuvent être corrigés en propageant les erreurs de couche en couche en commençant par la couche de sortie et en remontant, d'où le nom de rétropropagation. La performance du modèle MLP dépend non seulement du choix des variables, du nombre de couches cachées, de noeuds et de données d'entraînement, mais aussi des paramètres d'apprentissage tels que le taux d'apprentissage, le momentum contrôlant la variation des poids et le nombre d'itérations. Un MLP avec une seule couche cachée identifie la fonction non linéaire avec une précision moindre. Les réseaux avec plus de couches cachées risquent de surajuster les données d'entraînement. Le taux d'apprentissage et le momentum contrôlent la vitesse et l'efficacité du processus d'apprentissage.

3.3.2.2 Architecture du MLP :

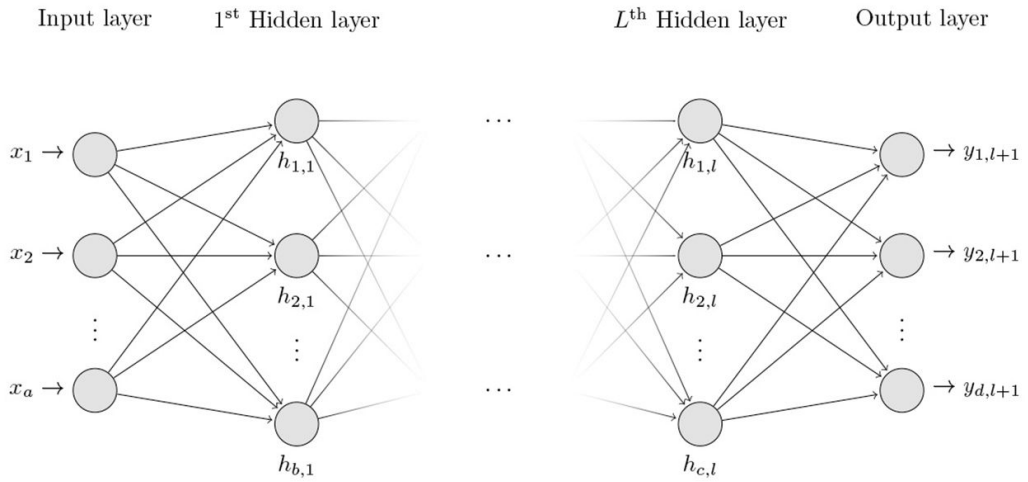

FIGURE 3.5 – Architecture du MLP

1. **Input Layer :** La couche d'entrée est la première couche du réseau de neurones. Son rôle est de recevoir les données brutes du problème étudié et de les transmettre aux couches suivantes sans traitement complexe. Chaque neurone de cette couche correspond généralement à une caractéristique (feature) des données d'entrée.
2. **Hidden Layer :** Les couches cachées se trouvent entre la couche d'entrée et la couche de sortie. Elles effectuent les calculs mathématiques du réseau neuronal à partir des poids, des biais et des fonctions d'activation. Ils visent à sélectionner les caractéristiques les plus pertinentes et de modéliser les interactions complexes entre les données.

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right) \tag{3.1}$$

ou :

- x_i : les valeurs d'entrée,
- w_i : les poids synaptiques,
- b : le biais (bias),
- f : la fonction d'activation,
- y : la sortie du neurone.

3. **Output Layer :** La couche de sortie est la dernière couche du réseau neuronal. Elle retourne le résultat final du modèle sous la forme d'une prédiction, d'une classification ou d'une valeur numérique. Le nombre de neurones dans cette couche dépend du type de problème que l'on veut résoudre : une seule sortie pour une régression, ou plusieurs neurones pour une classification à plusieurs classes.

3.3.2.3 Fonctionnement du MLP :

L'algorithme de rétropropagation est un algorithme d'apprentissage pour les perceptrons multicouches (MLP). Il se compose de deux phases : la propagation avant et la rétropropagation.

— **la propagation avant :** Lors de la propagation avant, les données d'entrée sont transmises à travers le réseau, et des prédictions sont générées. L'erreur entre les prédictions et les sorties souhaitées est ensuite calculée.[13]

— **rétropropagation :** Lors de la rétropropagation, l'erreur est propagée en sens inverse à travers le réseau, et les gradients. L'algorithme de rétropropagation est l'un des plus utilisés pour l'apprentissage des réseaux neurones. Il permet d'ajuster les poids des connexions du réseau, minimisant ainsi l'erreur entre les sorties prédites et les sorties attendues.[13] suivant plusieurs étapes :

1. **Initialisation des poids :** Les poids initiaux des connexions du réseau sont initialisés à des valeurs aléatoires. Ces poids déterminent la force des connexions entre les neurones et leur influence sur les sorties prédites du réseau.
2. **Propagation directe :** Les données d'entrée sont transmises à travers le réseau, et les sorties de chaque neurone de toutes les couches sont calculées. Ce processus est effectué séquentiellement de la couche d'entrée à la couche de sortie.
3. **Calcul de l'erreur :** Après avoir obtenu les sorties prédites par le réseau, l'erreur entre ces sorties et les sorties souhaitées est calculée. Cette erreur peut être mesurée à l'aide de différentes fonctions d'erreur, telles que l'erreur quadratique moyenne (EQM).
4. **Rétropropagation :**L'erreur est propagée en sens inverse à travers le réseau, et l'erreur est calculée pour chaque neurone de chaque couche. effectué à l'aide de la règle de la chaîne du calcul différentiel, qui permet le calcul du gradient de l'erreur par rapport aux poids.

5. **Mise à jour des poids :**Après le calcul des gradients de l'erreur par rapport aux poids, les poids des connexions sont mis à jour dans une direction minimisant l'erreur. Cette mise à jour s'effectue à l'aide de méthodes d'optimisation telles que la descente de gradient stochastique ou ses variantes.
6. **Itération (epochs) :**Les étapes précédentes sont répétées plusieurs fois (appelées époques) pour l'ensemble des données d'entraînement. Au cours des itérations, les poids des connexions sont mis à jour, ce qui réduit l'erreur, et le réseau s'adapte aux données fournies.

3.3.2.4 Paramètres et optimisation :

1. **Les paramètres du modèle :**
   - **Les poids (Weights)** w_i **:** Les poids contrôlent l'importance de chaque connexion entre les neurones.
     Ils sont ajustés pendant l'apprentissage afin de minimiser l'erreur du réseau.
     Le calcul d'un neurone dépend directement des poids :

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right) \tag{3.2}$$

   - **Le biais (Bias)** b **:** Le biais permet de décaler la fonction d'activation et d'améliorer la flexibilité du modèle.
     Il est ajouté à la somme pondérée des entrées :

$$z = (\sum_{i=1}^{n} w_i x_i + b) \tag{3.3}$$

2. **Les hyperparamètres :**
   - **Nombre de couches cachées :** Détermine la profondeur du réseau neuronal.
     - Peu de couches → modèle simple.
     - Plusieurs couches → meilleure capacité d'apprentissage mais risque de surapprentissage.
   - **Nombre de neurones par couche :** Contrôle la capacité de représentation du réseau.
     - Trop peu → sous-apprentissage.
     - Trop nombreux → coût computationnel élevé et overfitting.
   - **Fonction d'activation :** Permet d'introduire la non-linéarité dans le réseau. Les plus utilisées sont :
     - ReLU,

    - Sigmoid,
    - Tanh.
  - **Taux d'apprentissage (Learning Rate) :** Contrôle la vitesse de mise à jour des poids pendant l'apprentissage.
    - Trop grand → apprentissage instable.
    - Trop petit → convergence lente.
  - **Taille du batch (Batch Size) :** Nombre d'exemples utilisés avant une mise à jour des poids.
  - **Nombre d'époques (Epochs) :** Nombre total de passages sur l'ensemble des données d'entraînement.
3. **Méthodes d'optimisation :** L'optimisation consiste à ajuster les poids pour minimiser l'erreur du réseau.
  - **Descente de gradient (Gradient Descent) :** Méthode classique d'optimisation des poids.
    Mise à jour des poids :
    $$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial E}{\partial w} \tag{3.4}$$
    ou :
    - η est le taux d'apprentissage,
    - E est la fonction d'erreur.
  - **Backpropagation :** Algorithme principal utilisé dans les MLP pour propager l'erreur depuis la sortie vers les couches cachées.
  - **Optimisateurs modernes :** Les plus utilisés :
    - SGD,
    - Adam,
    - RMSProp,
    - Adagrad.
4. **Techniques de régularisation :**
  - **Dropout :** Désactive aléatoirement certains neurones pendant l'entraînement.
  - **Early Stopping :** Arrête l'apprentissage lorsque la performance cesse de s'améliorer.
  - **Régularisation L1/L2 :** Ajoute une pénalité sur les poids pour limiter leur amplitude.

3.3.2.5 Avantages et limites du MLP :

- **Avantages :**

- Les MLP ne possèdent que quelques paramètres ils peuvent donc être utilisés sans connaissances préalables et les algorithmes peuvent être mis en oeuvre facilement.
- Le processus d'apprentissage utilisé est adaptatif, ce qui permet aux MLP d'apprendre à trouver le solution directement issue des données modélisées.
- Les modèles hybrides qui utilisent à la fois des algorithmes d'apprentissage non supervisé et supervisé exister.

— **Limites :**

- Un réseau MLP nécessite un grand nombre de modèles et un grand nombre d'itérations pour apprentissage efficace.
- Définir le nombre de neurones et de couches dans la couche cachée d'un MLP est difficile.Par conséquent, de nombreux essais dans différentes conditions sont nécessaires pour trouver les meilleures combinaisons.
- Différentes configurations du réseau peuvent avoir des convergences différentes, en fonction de l'entraînement initial.

3.3.3 Federated Learning :

3.3.3.1 Introduction au Federated Learning :

1. **Définition :** définissons N propriétaires de données $\{F_1, F_2, \ldots, F_N\}$ qui souhaitent tous entraîner un modèle d'apprentissage automatique en consolidant leurs données respectives $\{D_1, D_2, \ldots, D_N\}$ Une méthode conventionnelle consiste à regrouper toutes les données et utiliser $D = D_1 \cup D_2 \cup \cdots \cup D_N$ Ce jeu de données global est ensuite utilisé pour entraîner un modèle centralisé appelé :M_{SUM} et on appelle la précision du modèle V_{FED}
   Cependant, dans le Federated Learning, les données restent stockées localement chez chaque propriétaire. Les différents clients collaborent alors pour entraîner un modèle fédéré noté : M_{FED} sans qu'aucun propriétaire F_i ne divulgue ses données D_i aux autres, la précision de ce modèle s'appelle V_{FED}
   L'objectif principal du Federated Learning est d'obtenir une performance du modèle fédéré aussi proche que possible de celle du modèle centralisé. Soit δ un nombre réel non négatif :
   $$|V_{FED} - V_{SUM}| < \delta \tag{3.5}$$
   - δ représente une petite marge d'erreur acceptable,
   - plus δ est faible, plus le modèle fédéré est proche des performances du modèle centralisé.

   Plus la différence entre les performances de M_{FED} et M_{SUM} est faible, plus l'algorithme du Federated Learning est efficace.

3.3.3.2 Fonctionnement du Federated Learning :

1. **Architecture générale :**

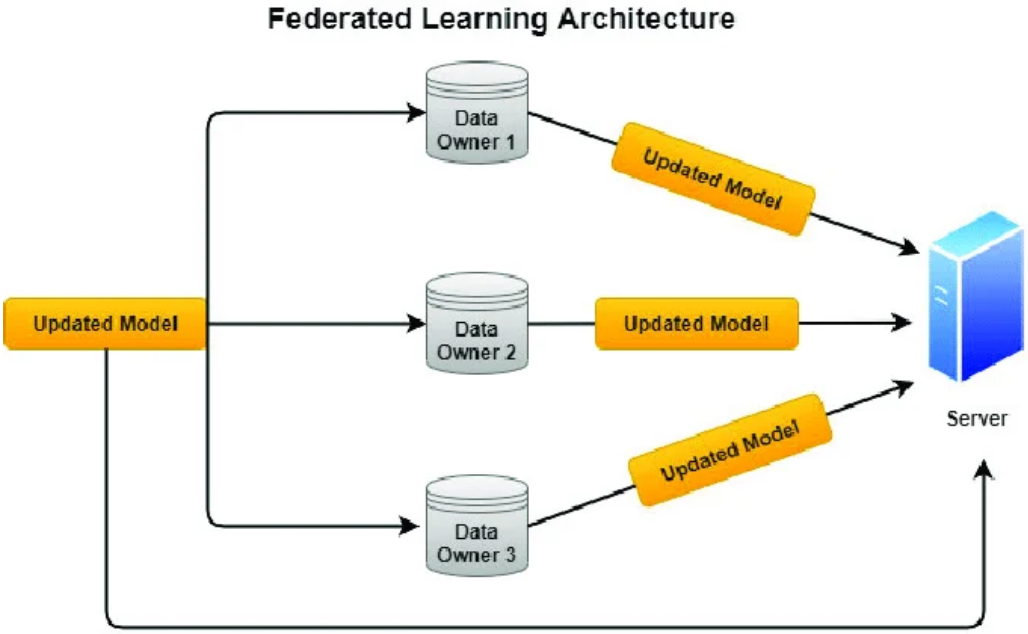

FIGURE 3.6 – Architecture du MLP

- **Serveur Centrale :** Le serveur central dirige tout le processus de Federated Learning. Son rôle principal consiste à :
  - Initialiser le modèle global
  - Envoyer le modèle aux différents clients
  - Recevoir les modèles mis à jour
  - Effectuer l'agrégation des paramètres
  - Redistribuer le nouveau modèle global.

  Le serveur ne reçoit jamais les données brutes des clients, mais seulement les paramètres ou gradients du modèle.

3.3.3.3 Algorithmes et architectures du Federated Learning :

3.3.3.4 Paramètres et optimisation :

3.3.3.5 Avantages du FL :

3.3.3.6 Limites et défis :

Chapitre 4

Implémentation et résultats

4.1 Introduction

4.2 Implémentation

4.3 Environnement de développement

4.3.1 Google Colab :

Dans le cadre présenté ci-dessus, Google a créé Colaboratory (également appelé Colab), un service cloud destiné à la diffusion de l'enseignement et de la recherche en apprentissage automatique [6]. L'environnement d'exécution fourni par ce service cloud est entièrement configuré avec les principales bibliothèques d'intelligence artificielle (IA) et offre également un GPU performant. Ce service Google est lié à un compte Google Drive et est gratuit.[14]

4.3.2 Visual Studio Code :

Visual Studio Code, souvent abrégé en VS Code, est un environnement de développement intégré (IDE) gratuit et open-source développé par Microsoft. Il s'est rapidement imposé comme l'un des IDE les plus populaires parmi les développeurs de logiciels, grâce à sa polyvalence, sa légèreté et son écosystème d'extensions robuste.[15]

4.4 Langage de programmation et bibliothèques

4.4.1 Python :

Est un langage de programmation puissant et facile à apprendre. Il dispose de structures de données de haut niveau et permet une approche simple mais efficace de la programmation orientée objet. Parce que sa syntaxe est élégante, que son typage est dynamique et qu'il est

interprété, Python est un langage idéal pour l'écriture de scripts et le développement rapide d'applications dans de nombreux domaines et sur la plupart des plateformes.
L'interpréteur Python peut être facilement étendu par de nouvelles fonctions et types de données implémentés en C ou C++ (ou tout autre langage appelable depuis le C). Python est également adapté comme langage d'extension pour personnaliser des applications.

4.4.2 Bibliothèques utilisées :

- **Numpy :** C'est une bibliothèque fondamentale dédiée aux calculs scientifiques en Python. Elle propose des structures de données puissantes ainsi que des fonctions très efficaces pour les opérations numériques, ce qui en fait une bibliothèque essentielle, notamment dans le domaine de l'apprentissage automatique et de l'analyse des données.
- **Pandas :** La bibliothèque logicielle open-source Pandas est spécifiquement conçue pour la manipulation et l'analyse de données en langage Python. Elle est à la fois performante, flexible et simple d'utilisation. Grâce à Pandas, le langage Python permet enfin de charger, d'aligner, de manipuler ou encore de fusionner des données. Les performances sont particulièrement impressionnantes quand le code source back-end est écrit en C ou en Python.[16]
- **Scikit-learn :** Scikit-Learn est une bibliothèque Python, libre et open source, destinée à l'apprentissage automatique. Initiée en 2007 et pilotée par Inria, elle rassemble plus de 2 000 contributeurs à travers le monde et compte près de 30 millions de visites sur son site en 2022, faisant d'elle le troisième logiciel libre de machine learning le plus utilisé au monde. En une quinzaine d'années, elle est ainsi devenue une référence en matière d'intelligence artificielle. Plus de 80 des data scientists confient l'utiliser quotidiennement en 2022.[17]
- **TensorFlow :** TensorFlow est une bibliothèque open source dédiée au calcul numérique, au machine learning à grande échelle, au deep learning et autres charges de travail de statistique et d'analytique prédictive. Ce type de technologie simplifie et accélère le travail des développeurs qui mettent en œuvre des modèles de machine learning en soutenant plusieurs tâches, dont l'acquisition des données, la livraison de prédictions à grande échelle et l'affinage des résultats futurs.[18]
- **PyTorch :** PyTorch est un cadre d'apprentissage profond open source basé sur un logiciel utilisé pour construire des réseaux neuronaux, combinant la bibliothèque de machine learning (ML) de Torch avec une API de haut niveau basée sur Python . Sa flexibilité et sa facilité d'utilisation, entre autres avantages, en ont fait le principal cadre de ML pour les communautés universitaires et de recherche.[19]
- **Matplotlib :** Matplotlib est l'une des bibliothèques Python les plus populaires pour la visualisation de données. Elle permet de transformer des données brutes en graphiques

lisibles et compréhensibles, notamment pour les Business Analysts qui souhaitent pouvoir représenter leurs données visuellement de manière claire et convaincante. [20]

FIGURE 4.1 – Outils Python - ML

4.5 Base d'apprentissage et de test : CICIoT2023 – TON$_{IoT}$ $^{\smile}$ BoT_{IO}

4.5.1 NSL-KDD :

L'ensemble de données KDDCUP'99 est largement utilisé pour la construction de systèmes de détection d'intrusion (IDS) [23] et pour l'évaluation des méthodes de détection d'anomalies. Outre la catégorie des intrusions normales (réseaux ne présentant aucune intrusion), les attaques sont classées en quatre catégories [17] : - Déni de service (DoS) : paralysie d'un réseau par saturation d'informations et de requêtes ; Accès distant à local (R2L) : accès non autorisé depuis une machine distante ; Accès utilisateur à distant (U2R) : tentatives d'accès à un compte utilisateur normal ; Sondage : intrusions de surveillance sur les réseaux.

KDDCUP'99 est un ensemble de données de référence en cybersécurité, créé en 1999 par la DARPA (Defense Advanced Research Projects Agency). Il a été utilisé pour le troisième concours international d'outils de découverte de connaissances et d'exploration de données (ITDOOT), dont l'objectif principal était la création d'un modèle d'intrusion réseau capable de classer les intrusions comme légitimes, malveillantes ou normales. L'ensemble de données comprend 4 898 430 échantillons, chacun contenant : 41 caractéristiques appartenant à l'un des 3 groupes suivants : - Caractéristiques de base, Caractéristiques de contenu ou Caractéristiques de trafic, elles-mêmes divisées en caractéristiques de trafic temporelles ou liées à l'hôte.
Dans cet ensemble de données, un point de données est soit normal, soit une attaque, et chaque attaque peut être étiquetée comme une attaque par déni de service (DoS), une attaque U2R, R2L ou une attaque de sondage.[21]

4.5.2 CICIoT-2023 :

a été créé par L'Institut canadien de cybersécurité afin de fournir un ensemble de données réel et complet pour l'environnement IoT. Ce jeu de données a été collecté grâce à un environnement de simulation physique réel comprenant divers dispositifs tels que des capteurs, des caméras intelligentes, etc. ; de multiples protocoles tels que DNS, HTTP, HTTPS et MQTT ; diverses attaques modernes ; et une plateforme d'exploitation réelle (serveurs, Wi-Fi, point d'accès, pare-feu et Wireshark). De plus, les attaques qu'il contient sont de véritables attaques générées à l'aide d'outils professionnels tels que Nmap, Hping3 et Metasploit. L'ensemble de ces outils a permis de générer un jeu de données riche contenant plus de 14 millions d'enregistrements, 46 caractéristiques et 34 attaques. Ce jeu de données contribue à sa fiabilité pour la construction de systèmes de détection d'intrusion robustes, notamment pour l'entraînement et l'évaluation des IDS[22]

4.5.3 BoT-IoT :

L'ensemble de données BoT-IoT a été créé en concevant un environnement réseau réaliste au sein du laboratoire Cyber Range de l'UNSW Canberra. Cet environnement intégrait un trafic réseau normal et un trafic provenant d'un botnet. Les fichiers sources de l'ensemble de données sont fournis dans différents formats : fichiers pcap originaux, fichiers argus générés et fichiers csv. Ces fichiers ont été classés par catégorie et sous-catégorie d'attaque afin de faciliter le processus d'étiquetage.

4.6 Implémentation de l'architecture proposée

4.6.1 Phase de prétraitement

4.6.2 Phase de création du modèle et d'apprentissage

4.6.3 Phase de prédiction et d'évaluation

4.7 Résultats, discussions et comparaison

4.8 Conclusion

Conclusion

Bibliographie

[1] A. Khraisat, A. Alazab, S. Singh, T. Jan, and A. J. Gomez, "Survey on federated learning for intrusion detection system : Concept, architectures, aggregation strategies, challenges, and future directions," *ACM Computing Surveys*, 2024.

[2] D. Torre, A. Chennamaneni, J. Jo, G. Vyas, and B. Sabrsula, "Toward enhancing privacy preservation of a federated learning cnn intrusion detection system in iot : Method and empirical study," *ACM Computing Surveys*, 2024.

[3] R. Lazzarini, H. Tianfield, and V. Charissis, "Federated learning for iot intrusion detection," *AI*, vol. 4, no. 3, pp. 509–530, 2023.

[4] A. Karunamurthy, K. Vijayan, P. R. Kshirsagar, and K. T. Tan, "An optimal federated learning-based intrusion detection for iot environment," *Scientific Reports*, 2024.

[5] S. Ghernaouti, *Sécurité informatique et réseaux*, 4th ed. Paris : Dunod, 2016.

[6] M. El Ajjouri, S. Benhadou, and H. Medromi, "Etude comparative des architectures de détection d'intrusion à base des systèmes multi agents," article scientifique.

[7] "Overview on intrusion detection systems for computers networking security," *Computers*, 2024. [Online]. Available : https://www.mdpi.com/2073-431X/14/3/87/pdf

[8] "Détection d'intrusions paramétrée par la politique de sécurité grâce au contrôle collaboratif des flux d'informations," researchGate. [Online]. Available : https://www.researchgate.net/publication/30514332

[9] "Advancing multi-class intrusion detection : A comparative evaluation of lstm and bi-lstm on class-imbalanced cic-ids-2017," *ResearchGate*, 2024. [Online]. Available : https://www.researchgate.net/publication/396148004

[10] Académie des sciences, "Intelligence artificielle et société," 2019. [Online]. Available : https://www.academie-sciences.fr/pdf/rapport/AI_G7_2019_FR.pdf

[11] L. De Matteis, S. Janny, S. Nathan, and W. Shu-Quartier, "Introduction à l'apprentissage automatique," *Revue 3EI*, 2022, disponible en ligne.

[12] M. U. Tobgé, "Détection distribuée d'anomalies dans les flux de données," Thèse de doctorat, Institut Supérieur d'Électronique de Paris (ISEP), Paris, France, 2022, directrice : Raja Chiky ; Co-encadrants : Yousra Chabchoub, Aliou Boly.

[13] K. Markov, “Multilayer perceptron with backpropagation, hdl coder, and fpga technology : An integrated approach for efficient neural network implementation,” *Problems of Engineering Cybernetics and Robotics*, vol. 80, pp. 13–22, 2023.

[14] T. Carneiro, R. V. Medeiros da Nóbrega, T. Nepomuceno, G.-B. Bian, V. H. C. de Albuquerque, and P. P. Rebouças Filho, “Performance analysis of google colaboratory as a tool for accelerating deep learning applications,” *IEEE Access*, 2018. [Online]. Available : https://www.researchgate.net/publication/328158184_Performance_Analysis_of_Google_Colaboratory_as_a_Tool_for_Accelerating_Deep_Learning_Applications

[15] S. Robert, “Visual studio code : l’éditeur pour les devops,” https://blog.stephane-robert.info/docs/developper/autres-outils/ide/visual-studio-code/, 2025.

[16] D. Cohen. (2026) Pandas : la bibliothèque python dédiée à la data science. Consulté le 2026. [Online]. Available : https://liora.io/pandas-python-data-science

[17] Fondation Inria. (2023) Scikit-learn : la bibliothèque française de l’intelligence artificielle. [Online]. Available : https://www.fondation-inria.fr/action_inria/scikit-learn-la-bibliotheque-francaise-de-lintelligence-artificielle/

[18] Databricks, “What is tensorflow ? machine learning framework,” https://www.databricks.com/blog/what-is-tensorflow, 2024.

[19] IBM, “Qu’est-ce que pytorch ?” https://www.ibm.com/fr-fr/think/topics/pytorch, 2024.

[20] Wild Code School, “Pourquoi et comment utiliser matplotlib avec python,” https://www.wildcodeschool.com/blog/pourquoi-et-comment-utiliser-matplotlib-avec-python, 2024.

[21] M. K. Ngueajio, G. Washington, D. B. Rawat, and Y. Ngueabou, “Intrusion detection systems using support vector machines on the kddcup’99 and nsl-kdd datasets : A comprehensive survey,” *IEEE Access*, 2022.

[22] W. A. H. Salman and C. H. Yong, “Overview of the ciciot2023 dataset for internet of things intrusion detection systems,” *Mesopotamian Journal of Big Data*, p. Article 4, 2025.

[23] “Classification and importance of intrusion detection system,” *ResearchGate*, 2020. [Online]. Available : https://www.researchgate.net/publication/340655192

[24] “Sciencedirect article,” *ScienceDirect*, 2025. [Online]. Available : https://www.sciencedirect.com/org/science/article/pii/S1546221825004862

[25] “Sciencedirect article,” *Computer Networks*, 2021. [Online]. Available : https://www.sciencedirect.com/science/article/pii/S1389128621005405

[26] “Sciencedirect article,” *Array*, 2023. [Online]. Available : https://www.sciencedirect.com/science/article/pii/S2405959523000334

[27] “Sciencedirect article,” *Ad Hoc Networks*, 2024. [Online]. Available : https://www.sciencedirect.com/science/article/pii/S1570870524001513

[28] “Sciencedirect article,” *Array*, 2025. [Online]. Available : https://www.sciencedirect.com/science/article/pii/S2405959525001742

[29] S. Serardi and M. E. H. Benhamouda, “Vers un système de détection d’intrusion dans l’internet des objets,” Master’s thesis, Université Ibn Khaldoun - Tiaret, 2020.

[30] M. B. S. Mlle AMIAR Thinhinane, “Realisation d’un ids a base d’un reseau lstm,” Master’s thesis, Université MOULOUD MAAMERI - Tizi-Ouzou, 2021-2022.

[31] “Principales cyberattaques dans l’iot,” *GlobalSign Blog*, May 25, 2023. [Online]. Available : https://www.globalsign.com/fr/blog/principales-cyberattaque-iot

[32] I. N. B. A. Wirianda, R. B. Huwae, and A. H. Jatmika, “Testing the effectiveness of intrusion detection systems (ids) snort, suricata, and zeek against syn flood attacks,” *BITE : Jurnal Bumigora Information Technology*, 2025, diterima : 19 Juni 2025, Direvisi : 13 Agustus 2025, Disetujui : 4 Desember 2025. [Online]. Available : https://journal.universitasbumigora.ac.id/bite/article/view/5226

[33] Python Software Foundation, “Documentation officielle de python 3,” https://docs.python.org/fr/3/, 2024, consulté le 19 avril 2026.

[34] OSSEC Project. (2025) About ossec. Consulté en 2026. [Online]. Available : https://www.ossec.net/docs/about.html

[35] IBM, “Machine larning,” 2026. [Online]. Available : https://www.ibm.com/think/topics/deep-learning

[36] Q. Yang, Y. Liu, T. Chen, and Y. Tong, “Federated machine learning : Concept and applications,” *ACM Transactions on Intelligent Systems and Technology*, vol. 10, no. 2, pp. 1–19, February 2019.